

Lego Bookers
SW3
A Well-Structured Program

Andreas Sebastian Hinge
Hifza Nasir

Benjamin Speyer Larsen
Jakob Thorøe

Goran Kirovski
Jonas Curran

21. december 2022



**AALBORG
UNIVERSITY**



**AALBORG
UNIVERSITY**

Software
Aalborg University
<http://www.aau.dk>

Titel:

En velstruktureret applikation

Tema:

Administration af Siddepladser

Projektperiode:

Vintersemester 2022

Projektgruppe:

SW3 Group 5

Deltager(e):

Andreas Sebastian Hinge
Benjamin Speyer Larsen
Goran Kirovski
Hifza Nasir
Jakob Thorøe
Jonas Curran

Vejleder(e):

Danny Bøgsted Poulsen

Oplagstal: 1

Sidetal: 59

Afleveringsdato:

21. december 2022

Abstract:

Denne rapport omhandler udviklingsprocessen af et system, hvis formål er at løse et problem angående siddepladserne på et af LEGO's kontorer. På kontoret er der ikke plads nok til alle der er ansat, så der bliver gjort brug af et "free-seating" system, hvilket leder til usikkerhed i forhold til tilgængelighed af pladser. Det system der blev udviklet i sammenhæng med projektet er en reserveringsapplikation, der giver dens brugere et overblik over pladserne på kontoret. Systemet deler sæderne op i zoner, hvor en bruger kan reservere pladser i zonen, enten for dem selv, eller for deres arbejdsteam. En bruger kan derudover søge efter deres medarbejdere for at finde ud af, hvem der er tilstede den pågældende dag.

Systemet er udviklet i Java med objekt-orienteret programmering, med yderligere brug af Spring som dets framework. Information angående zonerne og deres tilgængelighed, samt brugerinformation bliver gemt på en ekstern MySQL database.

Andreas Sebastian Hinge
-ahinge21@student.aau.dk-

Benjamin Speyer Larsen
-bsla21@student.aau.dk-

Goran Kirovski
-gkirov21@student.aau.dk-

Hifza Nasir
-hnasir21@student.aau.dk-

Jakob Thorøe
-jthoro21@student.aau.dk-

Jonas Curran
-jcurra21@student.aau.dk-

Indhold

1	Introduktion	1
2	Problem Definerings	2
2.1	Hvor har vi problemet fra?	2
2.2	Problemet	2
2.3	Rigt billede	3
2.4	Nuværende løsninger	4
2.5	Problemformulering	4
3	System definerings	5
3.1	Valg af System	5
3.2	BATOFF	6
4	Krav specifikation	8
4.1	MoSCoW	9
4.1.1	Must Have	9
4.1.2	Should Have	10
4.1.3	Could Have	10
4.1.4	Wont Have	11
5	Design	13
5.1	PAKT analyse	13
5.2	Objekt og Klasse analyse	15
5.3	Hændelsestabel	16
5.4	Adfærdsmønstre og Aktør	17
5.5	Navigationskort	19
5.6	Lo-Fi prototype	21
5.7	Hi-fi prototype	25
6	Implementering	32
6.1	Design-Strategi	32
6.2	Systemet i Angular	33
6.2.1	Hvorfor vi kasserede Angular Systemet	37
6.3	Systemet i Spring	38
7	Kode	40
7.1	Introduktion af Spring	40
7.2	Brugergrænsefladen	40
7.3	Funktionslaget	41
7.3.1	Funktionskald	42
7.3.2	Ændring af data	44
7.4	Modellaget	45

8	Testing	47
9	Diskussion	49
9.1	Resultat af Testing	49
9.2	Perspektivering til Arbejdsspørgsmål	50
9.3	Forbedringer på Systemet	52
9.4	Retrospektive Overvejelser	53
10	Konklusion	54
	Litteratur	55

1 Introduktion

Firmaet LEGO har et kontor med beliggenhed i Billund, hvor de har omkring 500 ansatte medarbejdere. På kontoret befinder der sig omkring 400 siddepladser, som de ansatte kan gøre brug af som de har lyst. Med andre ord, har de ikke et system til, at bestemme hvem der sidder hvor eller hvornår. For at afhjælpe problemet har de givet lov til, at ansatte kan arbejde hjemmefra, forbeholdt at de arbejder fysisk i en acceptabel grad.

Alle medarbejdere arbejder i grupper, og problemet opstår når en gruppe gerne vil arbejde sammen fysisk. De har ingen måde hvorpå de kan sikre sig med garanti, at der kommer til at være ledige pladser til alle gruppemedlemmer, når de ankommer til kontoret.

Dette projekt kommer til at gå ud på, at finde og udvikle en softwareløsning til problemet. Vi kommer til at anvende metoder og værktøjer som vi lærte under kurserne, "System Udvikling" og "Design og Evaluering af Brugergrænseoverflader", på det tredje semester af Aalborg Universitets Software uddannelse. Systemet vi kommer til at udvikle skal gøre brug af objektorienteret programmering, og til det, er der brugte forskellige frameworks til at udvikle et program som kan udfylde vores krav.

2 Problem Definering

2.1 Hvor har vi problemet fra?

Vi har fået kontakt til en medarbejder fra LEGO som sidder i den digitale afdeling. Vi fik præsenteret semesterprojektets emne, samt at vi gerne ville løse et problem med en IT løsning, og fik dermed dannet en problemstilling der omhandler et pladsorienteret problem på et kontor i sammenspil med vores kontaktperson.

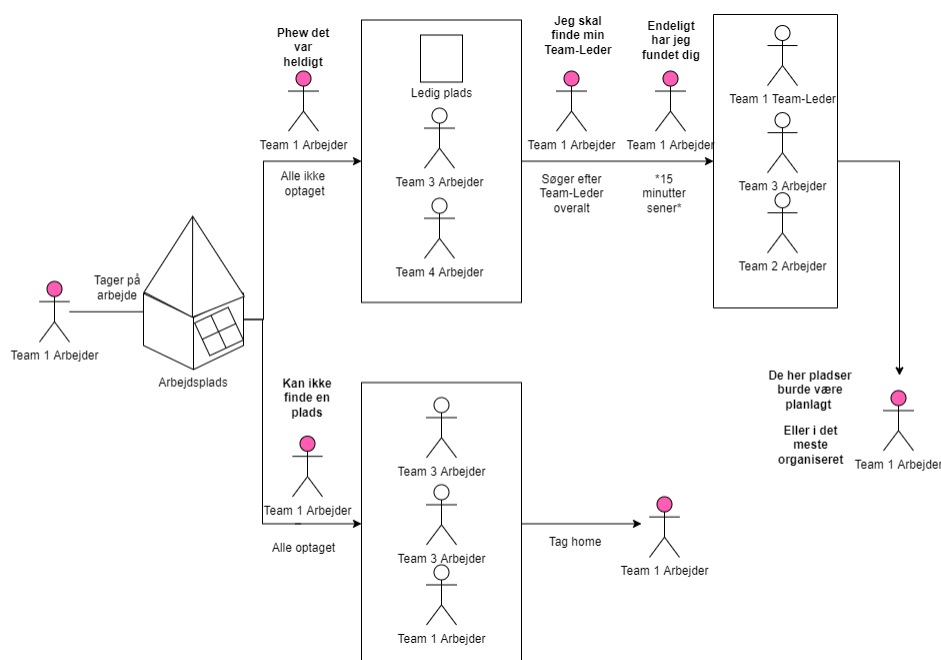
2.2 Problemet

LEGOs kontorer i Billund og København har begrænset siddepladser samt et free-seating system, hvor man ikke har en fast plads, og i princippet kan sætte sig hvor som helst. Kontoret er dimensioneret til at 80% af medarbejderne kan være på kontoret fysisk på samme tid. De har yderligere en "Best of Both" politik som foreskriver at medarbejderne skal være på kontoret 3 dage om ugen, men med mulighed for at arbejde hjemmefra omkring to dage om ugen. Problemet opstår når mere end 80% af medarbejderne møder fysisk op på de samme dage. Lego benytter pt. en primitiv løsning på problemet, hvor medarbejder har "hjemmezoner", så man har en form for uofficiel base på kontoret. Da der ikke er nogen definitiv planlægning sker det dog, at folk møder ind de samme dage. Det ville sige at man kan risikere at hjemmezonen er fyldt når man ankommer til kontoret, og at man så er tvunget til at finde en plads langt væk fra sine kollegaer. Derudover betyder udspreddningen af diverse teams, at man ofte ikke ved om éns kollegaer er på kontoret, eller hvor de sidder. Alle medarbejdere er delt op i afdelinger, underafdelinger og yderligere et hold. Disse bliver indenfor firmaet omtalt som "domains", "subdomains" og "teams". Man arbejder så oftest indenfor sit team, så det kan nogle gange være et problem hvis ikke det er nemt at finde sine teammedlemmer, eller hvis man ikke kan arbejde med dem fysisk.

2.3 Rigt billede

For at danne et overblik over situationen, har vi fremstillet et rigt billede, der beskriver to scenarier: ét, hvor en medarbejder finder en plads, og oplever ulemperne ved ikke at vide hvor en specifik person sidder, samt en anden situation hvor en medarbejder ikke kan finde en plads.

Et rigt billede bliver ofte brugt til at bekendtgøre både læser og forfattere med et problem område. Man beskriver en situation med relevante elementer og relationer der fremhæver et problem i en given situation. Ved at skabe et klart overblik over situationen, bliver det nemmere at danne et fokusområde, og efterfølgende, en system definition.^[1]



Figur 1: Et rigt billede af den nuværende situation

Først ankommer medarbejderen til kontoret, hvilket vises foroven i det første trin. Fra skiltringen af kontoret deles billedet op i to grene, hvor den øverste beskriver situationen når man kan finde en ledig plads, mens den nedre gren beskriver tilfældet af, når der ikke kan findes en plads. På den øvre gren, efter medarbejderen finder en ledig plads et arbitrært sted, har de behov for at diskutere noget med deres teamleder. Uden nogen idé om hvor de kunne side, begynder medarbejderen at lede efter dem, og efter et stykke tid, som kunne have blevet brugt på noget vigtigere, ender de med at finde dem. I den nedre gren, finder medarbejderen ikke en ledig plads, og ender med at tage hjem og arbejde der. I begge situationer skal medarbejderen bruge tid og energi, hver gang de skal finde enten en ledig plads eller deres vejleder.

2.4 Nuværende løsninger

På nuværende tidspunkt har de nogle primitive løsninger til problemet. Blandt andet har de oprettet uofficielle ”hjemme-zoner” for hver afdeling, så der i det mindste er en smule opdeling af de forskellige teams. Denne løsning hjælper på problemet, sådan at medarbejdere kan få plads sammen med deres kollegaer, men det samme problem opstår stadig da hjemme-zonerne ikke kan rumme alle medarbejdere i en specifik afdeling. For eksempel kan et større domain på 150 mennesker have en hjemme-zone med plads til kun 50 mennesker, hvilket kun forstørrer problemet for de teams. Alternativt kan medarbejdere under et mindre domain let finde pladser, sådan at alle kan sidde sammen.

Derudover prøver de enkelte teams ofte at blive enige internt om, hvilke dage de mødes på kontoret, og hvilke dage de arbejder hjemmefra. Disse interne aftaler sker oftest på et lavt og uofficielt niveau, og ikke blandt hele afdelinger, eller på et officielt niveau, så selv om der bliver aftalt at et team skal møde op og arbejde sammen er det aldrig sikkert at der er plads.

2.5 Problemformulering

Denne sektion opsummerer vores korrespondance med kontaktpersonen hos LEGO i form af en problemformulering, samt nogle arbejdsspørgsmål. Vi har valgt at lave disse yderligere arbejdsspørgsmål, da vi mener, at de kan give mulighed for en mere konsekvent arbejdsgang, siden vi ikke bare får et overordnet problem at løse, men at uanset hvor i processen vi er, har vi altid et mindre mål at sigte efter.

Hvordan kan en softwareløsning hjælpe LEGOs kontormedarbejdere med at formindske potentiel forvirring når det kommer til tilgængeligheden af siddepladser på deres kontor, samt at give mulighed for en måde hvorpå de kan finde deres teammedlemmer?

Vores arbejdsspørgsmål er følgende:

- Hvilken slags metode skal der gøres brug af for at give en mere strømlinet pladsfordeling? Skal man kunne reservere sin plads for en given dag, eller skal alle have faster pladser der evt. en gang imellem bliver byttet rundt?
- Hvordan skal medarbejderene kunne tilgå programmet? Skal det være en online service, eller et program man skal installere, enten på egen computer eller fælles computer?
- Hvilke design elementer skal der ligges mest fokus på? Gør vi eksklusivt brug af specifikke design principper, eller designes det på vores egen gennemtænkt måde?
- Hvordan sikrer man sig, at programmet fungerer som kunden ønsker? I hvor stor en grad skal testing køre? Skal der laves kvalitativ testing eller kvantitativ testing?

3 System definering

I dette afsnit vil vi diskutere hvordan systemet skal fungere i forhold til problemet. Med problemformulering som baggrund, har vores kontaktperson hos LEGO givet nogle retningslinjer til, hvad systemet skal kunne som minimum. Dette kommer vi til at anse som krav til systemet, og på basis af dem, vil vi forklare hvad systemet skal kunne. Dernæst vil vi lave en BATOFF analyse baseret på dette system, hvilket vi vil bruge som udgangspunkt til en konkret systemdefinering.

Ønske nummer et: Et system hvor medarbejderne hjemmefra kan booke/reservere en plads på kontoret.

Ønske nummer to: Et system hvor man nemt kan se hvor en specifik medarbejder eller et team sidder når de er på kontoret.

Ønske nummer tre: Systemet skal være nemt at bruge og helst kunne anvendes fra en mobiltelefon.

3.1 Valg af System

Efter vi fik fastlagt de basiskrav foroven, begyndte vi at diskutere hvorledes systemet i det hele taget skulle køre. Ét af projektets overordnede temaer er en objektorienteret tilgang til en problemstilling, hvor der er et krav om brug af programmeringssproget Java og en overordnet programstruktur der benytter sig af principper fra objektorienteret programmering. Vi har overvejet nogle forskellige idéer, der hver især har oprindelse i implementationer af forskellige IT midler.

Vi overvejede at programmere eksklusivt i Java, og holde det hele lokalt. For at alle brugere så ville kunne tilgå systemet, ville det være på én computer der placeres et sted på kontoret. Denne løsning ville gøre det nemt at finde ens kollegaer ved at søge efter dem på denne enhed, men ville gøre det svært når det kommer til plads-reservering: det vil blot digitalisere det, fremfor at fikse det, idet at flere medarbejdere møder op, end der er pladser til rådighed. En anden mulighed ville være at lave et program der skal installeres på alle arbejders enheder, som kommunikerer med en database der ville tillade dem at booke pladser og finde grupper. Denne metode har ulempen i det at alle arbejder skal have installeret den, da den kun ville kunne bruges hvis alle gør brug af den. Derudover, hvis vi alligevel skal have et lokalt program tilknyttet til en database, besluttede vi os for, at vi lige så godt kunne udvikle en webapplikation.

Ved at overveje disse forslag er vi blevet sikre på, at en webapplikation er den bedste løsning, både i forhold til fordelene det giver, men også de ressourcer vi har til rådighed. Det sidste af de ovenstående krav nævner, at systemet helst skal kunne anvendes fra en mobiltelefon. Det bliver ikke nævnt som værende et decideret krav, men nærmere en ekstra funktionalitet som øger brugervenligheden. Under alle omstændigheder, er det relativt simpelt at tilpasse programmet til brug fra mobiltelefoner med en hjemmeside, hvilket ultimativt har fastlagt en webapplikation skrevet ved at bruge forskellige frameworks og visse Java del, som værende vores valgte system.

3.2 BATOFF

En BATOFF analyse bruges til udarbejdelse af en systemdefinition. BATOFF er et akronym for seks ord der beskriver forskellige vinkler et problem eller et system kan skues fra. Ved at redegøre for, og analysere disse vinkler, dannes der en bedre forståelse for hvad systemet er, og hvad det skal kunne. Vi vil efterfølgende opsummere analysen med den endelige system definition. [2]

Betingelser

Applikationen vil blive udviklet af en gruppe universitetsstuderende fra AAU i samarbejde med en repræsentant fra LEGO, som kommer til at være en af de endelige brugere af systemet. Brugerne kommer generelt til at have et godt niveau af IT-forståelse, da de alle er fra LEGOs digitale afdeling, som må formodes at have en vis kendskab inden for IT.

Anvendelsesområde

Systemet kommer til at blive brugt til at reservere kontorpladser til medarbejdere der arbejder for LEGO. Systemet ville helst kunne tilgås fra en mobil telefon, så medarbejdere ville have muligheden for, at reservere kontorpladser inden de møder op på kontoret. Systemet gør det også muligt at vide på forhånd om der er ledige pladser tilbage, så man undgår at tage på arbejde og mangle en plads.

Teknologi

Da vi primært kommer til at køre projektet med objekt-orienteret programmering, kommer vi til at kode i Java. Systemet kommer til, at blive brugt af mange forskellige brugere på forskellige enheder, hvilket fra et udviklingsperspektiv betyder, at systemet skal være tilknyttet til en server. For at det skal kunne anvendes på en telefon udelukker ikke, at det ikke skal kunne bruges på en computer. Vi vurderer så, at den bedste måde at sætte systemet op på, er med en hjemmeside, der tilpasser sig selv alt efter hvilken slags enhed der anvender systemet. Derudover vil der blive gjort brug af en server til at gemme logininformation for den enkelte bruger, samt den data, der er relevant for, at systemet kan køre optimalt, blandt andet fuldt navn, status på reservation af plads, samt status på om pladser er taget eller ej.

Objekter

Medarbejder, Kontorplads og Teams.

Funktioner

Systemet skal have alle medarbejdere registreret, reservere en kontorplads til enkelte medarbejdere eller hele team hjemmefra, samt vise brugeren hvorhenne en bestemt medarbejder har deres arbejdsstation. Derudover skal der være et kort over kontoret, der viser om en plads er reserveret eller ej.

Filosofi

Systemet er hovedsageligt et redskab til at facilitere intern administration af pladsfordeling for alle brugere/medarbejdere. Derudover er det en måde hvorved brugeren kan finde andre medarbejdere uden fysisk at gå rundt og lede på hele kontoret.

Til sidst, vil vi opsummere vores BATOFF analyse i form af en konkret definering af, hvad vores system er, hvad det bruges til og hvem der kommer til at gøre brug af det.

Et system i form af en hjemmeside, der giver mulighed for reservation af kontorpladser for medarbejdere på et kontor, der ellers ikke har faste pladser. Systemets eneste brugere er kontorets medarbejdere, som kan reservere én eller flere pladser, afhængig af om de skal arbejde med deres teams eller bare have en plads til dem selv. Systemet kan derudover vide hvor individuelle personer sidder for dagen, hvilket gør det nemt at finde en specifik kollega, i tilfælde af at der er behov for det. Systemet kommer til at gøre brug af et login system, der midlertidigt ikke kommer til at inkorporere LEGOs eksisterende loginsystem som medarbejderene i forvejen gør brug af. En bruger i systemet kommer til at have et navn, brugernavn og kodeord til login og om de har en reservation eller ej, og hvis de har én, hvor den er. Systemet kommer til at have et kort over alle etager, hvilket vil inkludere alle kontorpladser for den givne etage, samt at kunne se, hvor der er ledige pladser henne.

4 Krav specifikation

I det forrige afsnit fik vi kort nævnt de funktioner og egenskaber som vi blev anmodet om at inkludere i vores system. Baseret på dem fik vi dannet en system definition, og i det næste afsnit vil vi sætte de samme krav op i en MoSCoW analyse, samt tilføje nogle flere krav som vi vurderer er vigtige. Vi kommer til at gøre brug af MoSCoW modellen da det er en måde at repræsentere éns krav på, der yderligere fortæller læseren hvor højt et krav er vurderet. MoSCoW står for "Must have, Should have, Could have og Wont have", hvor alle krav der placeres i "Must have" er krav der under alle omstændigheder skal inkluderes, mens resten er potentielle funktionaliteter der kan inkluderes, med en varierende grad af nødvendighed.

Vi kommer til at skelne mellem to slags krav: funktionelle og non-funktionelle krav. Den første indeholder krav der beskriver en funktion som systemet skal kunne udføre, mens den anden beskriver systemets egenskaber og hvordan det skal køre. Forneden ses de krav vi blev givet til at starte med, og i næste afsnit vil vi uddybe mere hvorfor og hvordan kravene vurderes, samt tilføje nogle nye krav der er blevet godkendt af vores kontaktperson.

Funktionelle krav:

- Et af de krav der blev prioriteret højest var, at systemet skulle skal gøre det muligt for medarbejderne at reservere en plads på kontoret, før de ankommer til arbejdspladsen.
- Det var meningen at teamlederen ville have mulighed for at reservere til hele gruppen, men efter at have snakket med vores kontaktperson, blev det ændret til at alle skulle have mulighed for at reservere til hele teamet.
- En hvilken som helst medarbejder skal kunne finde ud af, om en specifik kollega eller team er på kontoret den pågældende dag, samt kunne finde ud af hvor de sidder.

Non-funktionelle krav:

- Systemet skal være designet på en sådan måde, at en hvilken som helst bruger ville kunne anvende det uden megen besvær. Dvs. at der skal gøres mange overvejelser indenfor brugervenlighed.
- Derudover skal det kunne anvendes fra en mobiltelefon, så en bruger ville kunne reservere en plads fra hvor som helst, og uden at skulle tage sin computer frem.

4.1 MoSCoW

Med MoSCoW modellen som basis, kommer vi til at vurdere kravene alt efter hvor vigtige de er i forhold til programmets hovedsaglige formål.

I en MoSCoW model er der fire niveau af signifikans i forhold til hvilke funktioner der bedst tjener systemets formål, som vi kommer til at inddele hver af vores krav i:

- **Must Have** - Egenskaber og funktioner produktet skal have for at fungere på den ønskede måde.
- **Should have** - Egenskaber og funktioner der er vigtige for produktet, men som ikke afgør om det fungerer eller ej.
- **Could have** - Egenskaber og funktioner hvis relevans indenfor systemets hovedsaglige formål kan ses bortfra, men som ville forbedre det hvis de implementeres.
- **Wont have** - Egenskaber og funktioner der er blevet overvejet i en eller anden grad, men som under alle omstændigheder ikke skal eller kan blive implementeret.

Udover de krav vi fik fra firmaet, har vi tilføjet yderligere krav baseret på vores egne overvejelser og problem definitionen, herunder muligheden for at finde sine medarbejdere, samt viderbyggelse på nogle funktioner. Vi kommer til at opridsse alle krav, samt hvad de kommer til at indebære for systemet, men også hvorfor de er vurderet som de er.^[3]

4.1.1 Must Have

- **Reservering af kontorplads - Funktionelt krav**

Systemet skal give brugerne mulighed for at reservere en siddeplads til dem selv, eller at reservere pladser til flere personer, afhængig af éns team. Dette krav prioriteres højt da det er systemets hovedsaglige funktion

- **Se hvor folk sidder - Funktionelt krav**

Systemet skal give brugerne mulighed for at finde deres medarbejdere, hvis de har reserveret en plads på kontoret. Dette er yderligere ét af systemets grundlæggende funktioner, så det blev vurderet på samme niveau som det forrige.

- **Se ledige pladser - Funktionelt krav**

Systemet skal give brugerne mulighed for at finde ud af, hvor mange ledige pladser der er tilbage på kontoret samt hvor de er. Dette vil indebære en funktion der på én eller anden måde giver brugerne et overblik over kontoret, og uden det, ville de to ovenstående krav fungere mindre end optimalt, så det er blevet vurderet på samme niveau som dem.

- **Profil (Login system) - Non-funktionelt krav**

Systemet skal give brugerne mulighed for at kunne logge ind på systemet, således at der kan skelnes mellem forskellige medarbejdere. Efter en diskussion om hvorvidt profilering af medarbejderne skulle foregå, blev vi enige om at et login system med brugernavn og kodeord ville fungere bedst. Vi snakkede om at en implementation af andre slags profileringer kunne resultere i en slags identitetstyveri hvis ikke det blev gjort ordentligt, hvilket er grunden til at vi gør brug af et login system, da det stiller højere niveau af sikkerhed til rådighed, og er grunden til at kravet blev vurderet som værende et must have.

4.1.2 Should Have

- **Kan anvendes fra mobiltelefon - Non-funktionelt krav**

Systemet skal give mulighed for brugere at tilgå det fra en mobiltelefon. I mange situationer ville det være besværligt hvis man eksklusivt havde adgang til systemet igennem en computer. Meningen med mobiltelefoner i en professionel sammenhæng er, at give deres brugere adgang til sådanne systemer i situationer hvor det ellers ville være umuligt. Vi diskuterede om dette skulle være et must have eller should have, men endte med at placere den hernede, da vi tænkte at nem adgang til programmet ikke var essentielt for selve programmet. Derfor vurderer vi det som et should have, men med top-prioritet af ekstra egenskaber systemet kan få implementeret.

- **Sektionering af domains - Funktionelt krav**

Systemet skal give brugerne mulighed for at søge efter domains eller subdomains, fremfor at skulle søge efter navne, i tilfælde af at de har behov for at finde en medarbejder. Dette betyder at man ville kunne søge efter titler, såsom "Digital design" og dermed finde ud af hvor folk under Digital design sidder. Det er en viderebygning af søge systemet som blev beskrevet i must have, men med en egenskab der ikke er nødvendig for, at systemet kan køre optimalt. Selvom det ikke er nødvendigt at have det med, ville det give brugerne flere muligheder som vi har vurderet som værende væsentligt relevante, og er grunden til at det vurderes som en should have.

4.1.3 Could Have

- **Sikkerhed - Non-funktionelt krav**

Systemet skal have en hvis mængde sikkerhed, der vil gøre det svært at begå identitetstyveri. I tilfælde af at nogle får uautoriseret adgang til en brugers login, ville man kunne reservere en plads for dem eller afreservere den de har. Et systembrud kunne skabe kaos og forvirring, men ville ikke have større konsekvenser end blot en midlertidig ulejlighed. Sikkerhed er dog alligevel ikke noget der skal ses helt bortfra, så vi har placeret det herunder i could have, som noget vi kunne have med hvis lejligheden viser sig.

- **Dynamisk opereret layout - Funktionelt krav**

Systemet skal kunne give speciel adgang til visse brugere, som ville kunne ændre på layoutet af de visuelle elementer. Programmet kommer til at inkludere et kort over kontoret, og dets pladser, og i tilfælde af at ændringer i kontorets layout skulle ske, eller at nogle pladser bliver ubrugelig af forskellige årsager, ville denne "superbruger" eller administrator, kunne ændre på systemets korts layout, så det matcher med kontorets nuværende tilstand. Vi nåede frem til, at dette ikke var en del af systemets oprindelige formål, men at det kunne give god mening at implementere det alligevel, så dens relevans blev vurderet som værende en del af vores could have.

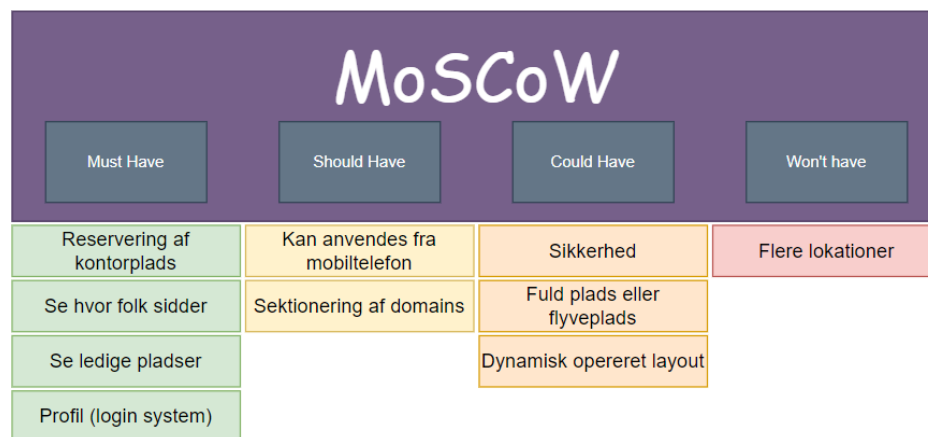
- **Fuld plads eller flyverplads - Funktionelt krav**

Systemet skal kunne give brugerene mulighed for at reservere flere slags pladser, fremfor bare en enkelt slags. De har ofte folk der f.eks. kun ankommer til kontoret for at deltage i et møde, og i sådan et tilfælde giver det ikke meget mening for dem at reservere en kontor plads. Hvis de så i stedet kunne reservere en såkaldt "flyverplads" som ligger ude for de her hjemmezoner som firmaet har, hvor de så kunne opholde både dem selv og deres ting. Vi har vurderet at det ikke er et vigtigt problem, der skal løses indenfor de første iterationer af programmet, så den bliver placeret her under could have.

4.1.4 Wont Have

- **Flere lokationer - Non-funktionelt krav**

Systemet skal kunne dække flere lokationer indenfor firmaet, og give mulighed for reservation indenfor alle inkluderede arbejdspladser. Dette er en egenskab som ville give god mening at implementere, men grundet at vores kontaktperson arbejder på kontoret i Billund, har vi ikke nok information om kontoret i København til at lave en velfungerende udgave af systemet egnet til det. Siden det kunne ende med at være to forskellige systemer, gav det ikke mening at implementere det på nuværende tidspunkt, og blev dermed vurderet som et wont have.



Figur 2: MoSCoW

Foroven ses en visuel repræsentation af vores MoSCoW model hvor både krav og vurdering nemt kan ses. Næsten alle krav der blev givet af firmaet en del af must have, siden de er byggestenene for hvad programmet burde kunne gøre. De krav vi selv er kommet med er mere fokuseret på funktionaliteten af programmet, og har derfor generelt en lavere prioritering i den endelige vurdering af kravene.

5 Design

I denne sektion kommer vi til at udarbejde et design til vores system baseret på hvad der hidtil er blevet undersøgt, samt med løbende feedback fra vores kontakt person. Vi kommer til først at foretage en PAKT analyse for at definere vores system i direkte kontekst med vores brugere, samt for at danne et bedre overblik over hvem der gør brug af systemet. Vi vil derudover skabe et konkret overblik over hvem brugerne er ved at definere hvordan deres opførsel er i forhold til systemets funktioner, med hjælp fra en hændelsestabel og en objekt-og klasse analyse, der kommer til at give en oversigt over hvordan objekterne interagerer med systemet. Dernæst vil vi fremstille et navigationskort for at vise hvordan en bruger vil kunne tilgå de forskellige funktioner som systemet tilbyder. Vi vil så gennemgå alle iterationer af vores lo-fi prototyper af systemet, med fokus på hvorfor visse ændringer blev gjort, hvorefter vores endelige design vil blive vist i form af en hi-fi prototype.

5.1 PAKT analyse

PAKT er et framework der kan blive brugt i sammenhæng med at designe interaktive systemer. Det indholder et menneskecenteret design fokus ved at beskrive et system fra fire forskellige vinkler med hensyn til brugeren: personer, aktiviteter, kontekst og teknologier. [4]

Personer

Systemets hovedbrugere er ansatte hos LEGO i deres kontor i Billund. På kontoret gør brugerne brug af computere til at udføre arbejdsopgaver, både fysisk på kontoret og når der skal arbejde hjemmefra, så meget af det sker digitalt. Baseret på hvad vi har fået at vide, og den kontekst hvori brugerne arbejder, kommer systemet ikke til at inkludere muligheder for at øge tilgængeligheden for folk med fysisk eller psykisk handicap. Derudover siden brugerne er omringet af teknologi på en daglig basis, kommer systemet ikke til at inkludere nogle form for vejledning eller en support side, men simple hjemmeside virkemidler, såsom store knapper der nemt kan ses samt tilhørende tekst til hvad knappen gør og navigation mellem sider i form af links der siger præcis hvilken sider den leder hen til.

Aktiviteter

Ved brug af systemet kan brugerne reservere pladser på kontoret, samt vide om en specifik person er tilstede en given dag eller ej. Hvis hele kontoret gør brug af dette system, vil det øge muligheden for en mere fokuseret arbejdsgang, da det kommer til at være nemt at finde ud af om der skal arbejdes fysisk eller ej. Hvis dagens arbejdsplads er indforstået, tillader det medarbejderne at fokusere mere på deres arbejde, fremfor deres arbejdsmiljø. Hvis ellers der er behov for korrespondance mellem teams, kan en bruger nemt finde ud af om det andet team er tilstede fysisk eller om de arbejder hjemmefra, ved at lave en søgning på enten én af deres navne eller team.

Systemet giver foreløbig kun mulighed for disse to funktionaliteter, og på nuværende tidspunkt ved vi ikke hvilken af disse to funktioner der kommer til at blive anvendt mest. Det hovedsaglige formål systemet opfylder er mulighed for reservation af pladser, men det betyder ikke nødvendigvis at det er den funktion der bliver brugt oftest. Baseret på data om hvilken funktion brugerne anvender mest, vil den funktions side agere som en startside som

brugerene først ser, når de logger ind.

Kontekst

Kontorets free-seating system som det ser ud uden brug af systemet fungerer i praksis kun på grund af kontorets tilgang til hjemmearbejde. Da kontoret er for småt til at alle medarbejdere kan være tilstede på samme tid, var det en hurtigste, og bedste løsning de kunne finde på. Hvis systemet bliver implementeret sådan at hele kontoret begynder at benytte det, afskaffer de sig i princippet med deres free-seating system, samt at det kan give mere kontrol over hvem der arbejder hjemmefra, og hvornår de gør det.

I forhold til den enkelte bruger, skal de også finde et tidspunkt at gøre brug af systemet på, og det enkelte team skal derudover måske være enige i reservering af en plads, fremfor at én person træffer beslutningen. Måden det umiddelbart kommer til at fungere på er, at efter et bestemt tidspunkt på dagen bliver alle reservationer nulstillet. Så vinduet en bruger har at reservere i kommer til at være afhængig af hvornår det nulstilles til hvornår end alle pladser bliver taget. Dette kan have betydning hvis en bruger eller et team er sent på den i forhold til reservationen, og gerne vil have en så hurtigt som muligt. I dette tilfælde, og for at undgå frustration under reservering, skal den funktion designes og implementeres sådan at den kan blive udført uden besvær og indenfor meget kort tid.

Teknologier

Systemet skal helst give brugere nem og hurtig adgang til dets funktioner, og den nemmeste måde det kan blive indført på er igennem implementation af systemet som en app på brugerens mobiltelefoner. På den måde vil en bruger kunne nemt tilgå systemet, og såfremt at systemet kører optimalt og hurtigt, vil udførelse af funktionaliteter heller ikke være til besvær.

Hvis systemet ikke kan implementeres som en app, vil det lige pludselig blive enormt besværligt at få adgang til det. Hvis man skal hurtigt ud af døren bliver det besværligt at hive sin laptop frem hver gang man skal reservere en plads. Derudover kan det heller ikke som sådan bliver gjort hvis man er under transit, på vej hen til arbejdspladsen. For eksempel, hvis man sidder i en bus, er det i høj grad nemmere at gøre brug af en telefon, fremfor en laptop.

I forhold til bruger anvendelse, er det ækvivalent med en standard hjemmeside. Brugeren interagerer med systemet med en mus eller en touchscreen, og gennem deres input, bliver brugeren givet output af systemet i form af tekst der giver information om brugerens nuværende status. Dette kan være om en reservation kan lade sig gøre, om deres team har reserveret pladser eller resultat på brugerens søgning.

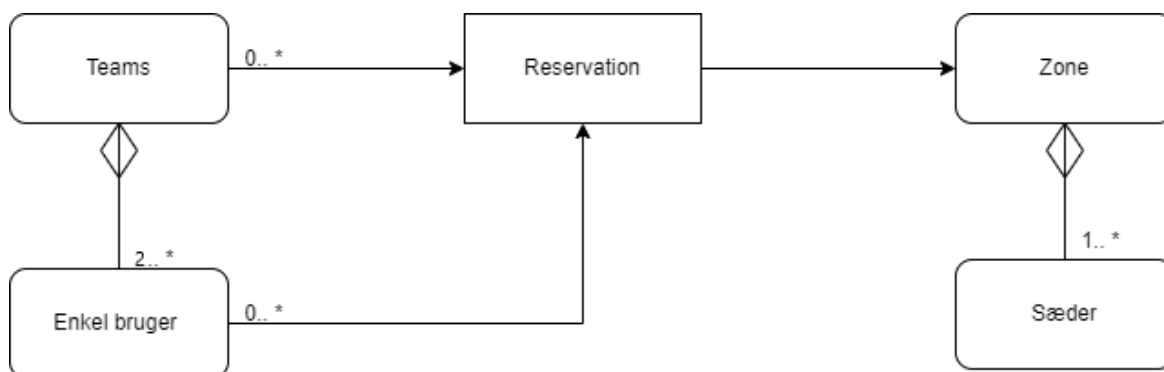
Ved at analysere vores system igennem en PAKT analyse er vi kommet frem til, at er den største egenskab vore system skal have er en simplicitet som enhver kontorarbejder ville kunne forstå. Brug af systemet skal ikke være besværligt, og så hurtigt som muligt. Derudover ville den have rigtig godt af en version der er tilegnet til anvendelse igennem mobiltelefoner.

5.2 Objekt og Klasse analyse

For at analysere problemet videre og forstå begivenheder, kigger vi på hvilke fysiske og abstrakte objekter der er, samt klasser der kan beskrive ensartede objekter. For at finde objekter, kigger vi på det adfærdsmæssige identiteter, som i vores problem er følgende: **Bruger**, **Teams**, og **Zone**

Objekterne foroven er skrevet i et klassediagram - som ses forneden i figur 3 - for at forstå strukturen mellem klasserne. De objekter vi konkret skal forholde os til i forhold til funktionaliteten af programmet, er de enkelte brugere og deres tilhørende team, samt hvilke zoner de befinder sig i. Sæderne tilhører ikke en særskilt klasse, men er en del af zone klassen. En bruger er den del af et team, og igennem en af dem bliver en reservation lavet. Den reservation påvirker så en zone, som består af et antal af sæder.

En hierarkisk struktur kan ses i bruger, reservation og zoneklassen[2]. Den angiver, at en bruger kan lave så mange reservationer som de har lyst til, hvor det ikke er påkrævet at de skal lave mindst én: en bruger skal have mulighed for at arbejde hjemmefra, hvilket betyder at de nogle gange ikke har brug for en reservation, og det samme gælder for teams. Et team har ikke et fast antal medlemmer, så i klassediagrammet skal et team blot bestå af mindst to brugere. Det samme gælder tildels for zonerne, da de heller ikke har et bestemt antal pladser de skal have, samt at en zone i princippet godt kan nøjes med at have en enkelt plads.



Figur 3: Klasse diagram

5.3 Hændelsestabel

Forneden ses en hændelsestabel, der beskriver de begivenheder der kan ske i systemet i relation til objekterne. Vi har lavet denne tabel for at danne en konkret sammenhæng mellem objekterne og funktionerne de kan udføre. Udover de førnævnte objekter og klasser, har vi tilføjet et ekstra objekt for at repræsentere en vigtig del af systemet.^[2]

- En bruger ville kunne logge ind på systemet, og reservere en plads i en vilkårlig zone, samt afreservere den i hvis det skulle blive nødvendigt. Derudover har brugeren valget mellem at gøre det til en enkeltreservation eller en teamreservation.
- Et team har funktionelt set ikke mulighed for, at reservere pladser. Dette er fordi et team er en samling af brugere, og er dermed ude af stand til at reservere en plads selv. Men et team er samlet set i stand til, at afreservere en plads hvis de har brug for det.
- En zone kan ikke udføre nogle funktioner selv, men er tilbøjelig overfor, at blive reserveret af en bruger. Når dette sker, vil en zone justere dens ledige pladser til at akkommodere for de nye brugere der nu har en reservation hos den.
- En administrator er lidt af en teoretisk tilføjelse. Vi har før snakket om, at en administrator ville være en bruger med ekstra privilegier, blandt andet brugeroprettelse. Vi følte at det var vigtigt at nævne brugeroprettelse, da det er en essentiel del af hvilket som helst system, der gør brug af login.

	Enkelt Reservation	Teamreservation	Afreserver	Blive Reserveret	Brugeroprettelse	Login
Bruger	X	X	X			X
Team			X			
Zone				X		
Administrator					X	

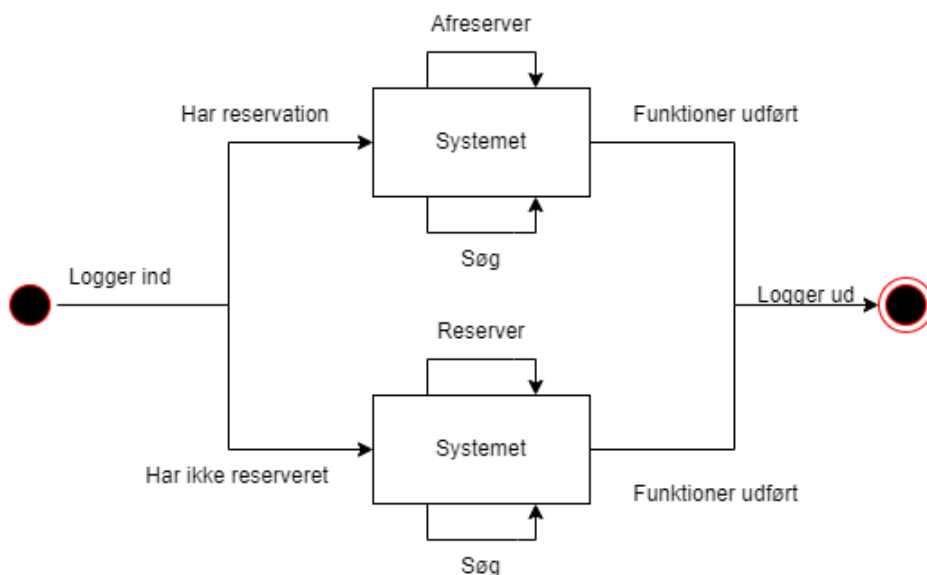
Figur 4: Hændelsestabel

5.4 Adfærdsmønstre og Aktør

I denne sektion, vil vi kigge på adfærdsmønstre af brugere, for at se hvordan de interagerer med systemet, hvilket vi kan se ved at lave et tilstandsdiagram over brugeren. Vi kommer til at kigge på brugerene og teams, og efter at have defineret, hvad objekter er i stand til, overvejer vi også aktørens motivation under de givne omstændigheder, samt eksempler på deres adfærd.

Bruger

Vores tilstandsdiagram i figur 5 illustrerer brugerens adfærd. Først indtaster brugeren deres brugernavn og adgangskode til det enkle formål at logge ind på systemet. Afhængig af om brugeren har en reservation eller ej, får de i princippet to udgaver af systemet: en hvor de kan reservere en plads, og en anden hvor de kan afreservere deres plads, hvor de i begge udgaver kan søge efter medarbejdere som de har lyst. Når en bruger så har færdiggjort deres brug af systemet, kan de blot logge ud, hvilket putter brugeren i deres slutstat i tilstandsdiagrammet.



Figur 5: Tilstandsdiagram af bruger

Vi vil nu overveje brugerens adfærd når de skal gøre brug af systemet, ved at definere deres formål for hvorfor de gør brug af systemet, hvilke egenskaber de har i forhold til systemet og et eksempel på typisk brugeradfærd i systemet.

Formål: Medarbejderen kan logge ind med brugernavn og adgangskode for at reservere pladser på kontoret på forhånd. Derudover, hvis de har behov for det, kan de søge efter deres medarbejdere for at se, om de er tilstede fysisk, og i så fald, hvor de sidder.

Egenskaber: En bruger har et unikt login til systemet, samt et team der kan ses på en af systemets undersider. De har to tilstande: med reservation, og uden reservation.

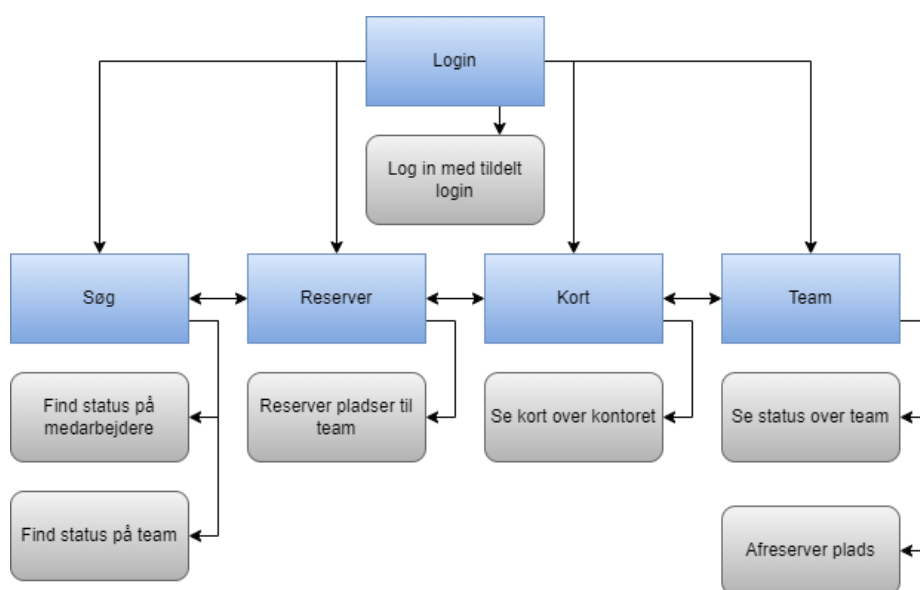
Eksempel 1: En bruger har en given dag mulighed for, at vælge mellem om de vil arbejde hjemmefra eller ej. Af en eller anden vilkårlig årsag, skal de arbejde fysisk, men vil sikre sig på forhånd at de har en siddeplads inden de tager hjemmefra. De logger ind på systemet og ser, at ingen af deres teammedlemmer har reserveret pladser, og siden de har aftalt at møde sammen den dag, vælger brugeren at foretage en teamreservation. De sikre sig efterfølgende så, om resten af teammedlemmerne fik registreret den samme reservation.

Eksempel 2: En bruger er på et given tidspunkt usikker på et vilkårligt emne. De ved med sikkerhed at denne specifikke som de kender er klogere på emnet end de er, så de laver en søgning på denne anden brugers navn, for at finde ud af, om de har en reservation denne dag. Det viser sig at de har én, så brugeren tager hen til den zone de er reserveret i, for at løse problemet.

5.5 Navigationskort

Et navigationskort er en visuel repræsentation af hvordan en bruger kan navigere rundt på siderne af en webapplikation, og på figur 6 ses vores udgave. Siderne er repræsenteret af de blå firkanter, og i vores model er funktionerne og handlinger derudover også vist i form af grå firkanter. Hvis en blå firkant peger på en anden af samme slags, betyder det at den anden side kan tilgås af den første, hvorimod en pil der går i to retninger repræsenterer at man kan tilgå begge sider fra hinanden. En pil der har oprindelse i en blå firkant og slutter i en grå, repræsenterer at funktionen eller handlingen eksisterer på denne specifikke hjemmeside.

Med funktionerne tilstede i visualiseringen af hele hjemmesiden, giver det lidt perspektiv til ikke bare hvordan man kommer hen til siderne, men også hvordan man kommer hen til funktionerne. [5]



Figur 6: Navigationskort

Når man forsøger at komme ind på en side på systemet, tjekker den om man er logget ind. Til dette eksempel siger vi, at dette er første gang systemet er blevet tilgået af denne bruger. Derfor, uanset om de prøver at tilgå nogle af de andre sider via hjemmesidens URL, bliver de sendt til login siden. Denne side tjener blot det enkelte formål at validere om en bruger har korrekt login information. Der eksisterer ikke på systemet hverken en signup side eller funktion der lader en ny bruger lave et login, da det kun bliver anvendt af firmaets ansatte, så måden hvorpå man får tildelt et login ville være igennem firmaet. Efter systemet har valideret brugerens login information, ville de i princippet blive sendt til den rigtige startside, men som beskrevet i PAKT analysen, er startside ikke - og kan ikke - besluttes på nuværende tidspunkt. Derfor er der foreløbige pile der går fra login siden ud til alle de andre sider.

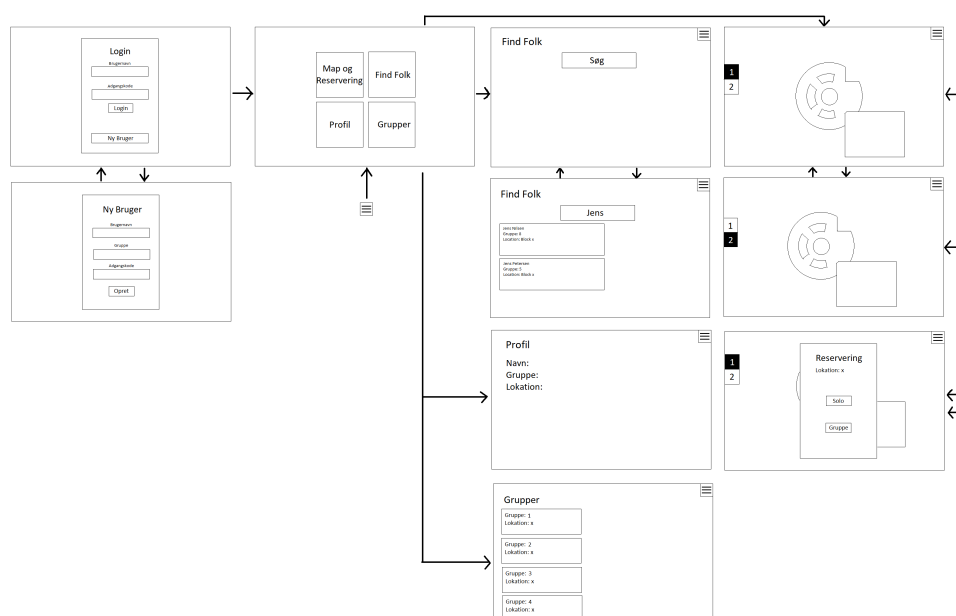
Uafhængig af hvilken side man lander på, ville de andre tre tilgås nemt via en form for navigationsfunktion. I mellem de fire sider ses tre pile der peger begge veje, hvilket betyder

at man f.eks. kan komme hen til "Team"siden fra "Reserver"siden, men for at få adgang til funktionerne, skal man være på den specifikke side. Hvis man vil søge efter sine medarbejdere, eller et team nummer eller team navn, skal man navigere hen til "Søg"siden, hvorimod hvis man nemt vil finde status over sit eget team, kan man blot navigere til "Team"siden, hvor éns teammedlemmers navne og om de er reserveret automatisk bliver fundet frem. Man kan derudover også afreservere sin og sit teams plads, men hvis man gerne vil reservere én først, skal man hen til "Reserver"siden, hvor man vil kunne se de zoner i kontoret der er ledige, hvor mange af dens pladser der er ledige, og ud fra det, bestemme hvilken plads man vil reservere for dagen. Men hvis man bare vil se et kort over kontoret uden informationen der tilhører reserveringsfunktionen, kan man tage hen til "Kort", der kun indeholder et rent kort over kontoret.

Systemets navigation er designet så enkelt som muligt baseret på vores PAKT analyse. Vi vil undgå frustration og forvirring under brugerens udførelse af funktionerne, og hvor det er relevant i et navigationskort, er hvordan og hvorfra man kan tilgå de forskellige sider. Jo mere simpelt navigationen mellem siderne er, jo mere tid kan dedikeres til udførelsen af sidernes funktioner.

5.6 Lo-Fi prototype

Efter at vi fik fastlagt funktionerne, og hvor de skulle tilgås af brugeren, fik vi dannet vores udkast til et design af systemet i form af nogle simple lo-fi sketcher. Denne tilgang er utrolig nyttig da den er enkel, hurtig og effektiv og uanset tegningens kvalitet kan der ofte stadig skues idéer om designet. For at se hvordan de navigationer vi oprettede ovenfor vil se ud, tegnede vi vores grove skitser af systemet på en tavle, og fik dem gentegnet i et digitalt rum. Der kan argumenteres for, at de så ikke længere er lo-fi sketcher, men vi valgte at gentegne dem digitalt for at nemmere kunne visualisere det for os selv, for firmaet og for læseren. De blev gentegnet ved hjælp af Figma[6] - et webprogram der bruges til at designe interaktive grænseflade prototyper. Når vi får feedback fra firmaet tilpasser vi lo-fi prototypen til bedre at visualisere og opfylde deres ønsker for systemet, og bliver ved med denne proces indtil firmaet er tilfredse med layoutet. [7]

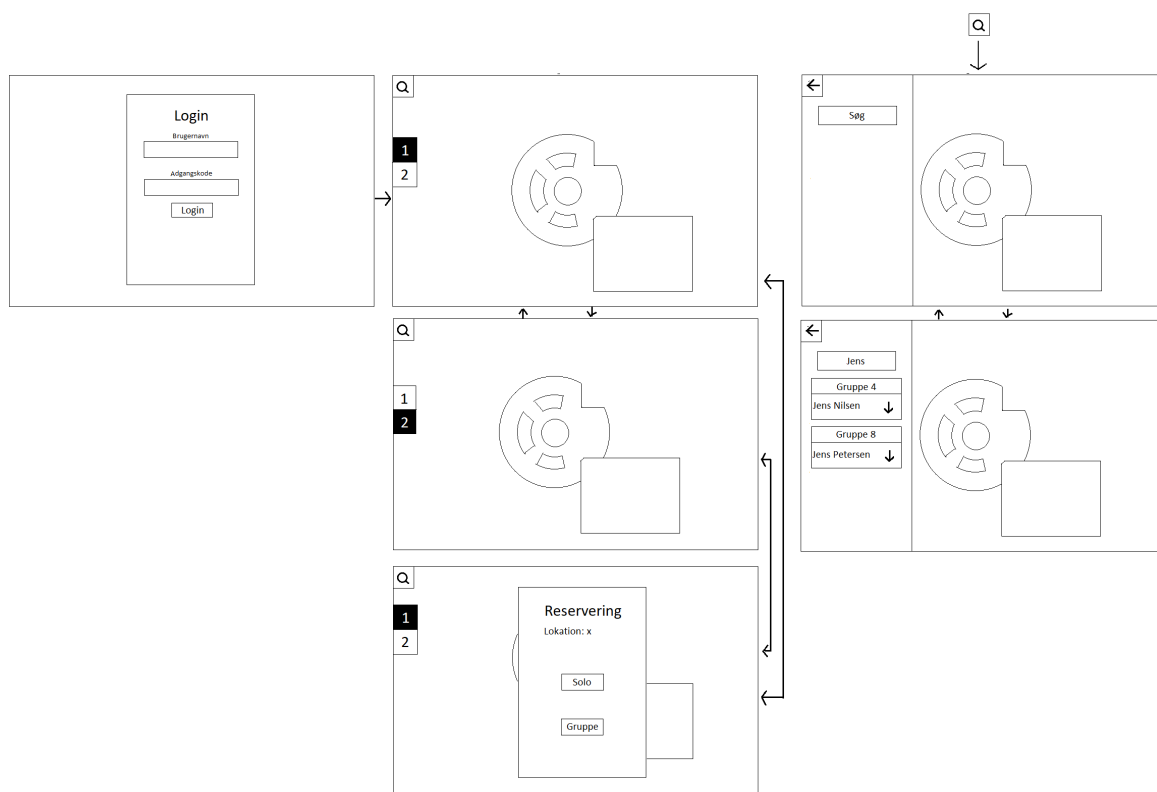


Figur 7: Lo-Fi 1

I figur 7 kan man se det første udkast til en lo-fi prototype af systemet. Øverst til venstre ses en login/signup side, der efter at have godkendt brugerens login leder dem hen til en navigationsmenu. Dette var vores første idé til hvordan en bruger kunne navigere rundt på hjemmesiden, og man kan komme hen til den fra hvilken som helst anden side, blot man trykker på "burger menu" ikonet, der er tilstede på alle undersider. De fire sider og funktioner man kan tilgå i denne version af systemet er "Kort og Reservering", "Find Folk", "Profil" og "Grupper". "Kort og Reservering" lader brugeren se et kort over kontoret, samt reservere en plads, og de sker begge to på den samme side, fremfor at være separeret i to sider. "Find Folk" er en søgefunktion hvor brugeren kan søge efter en specifik bruger eller et specifikt

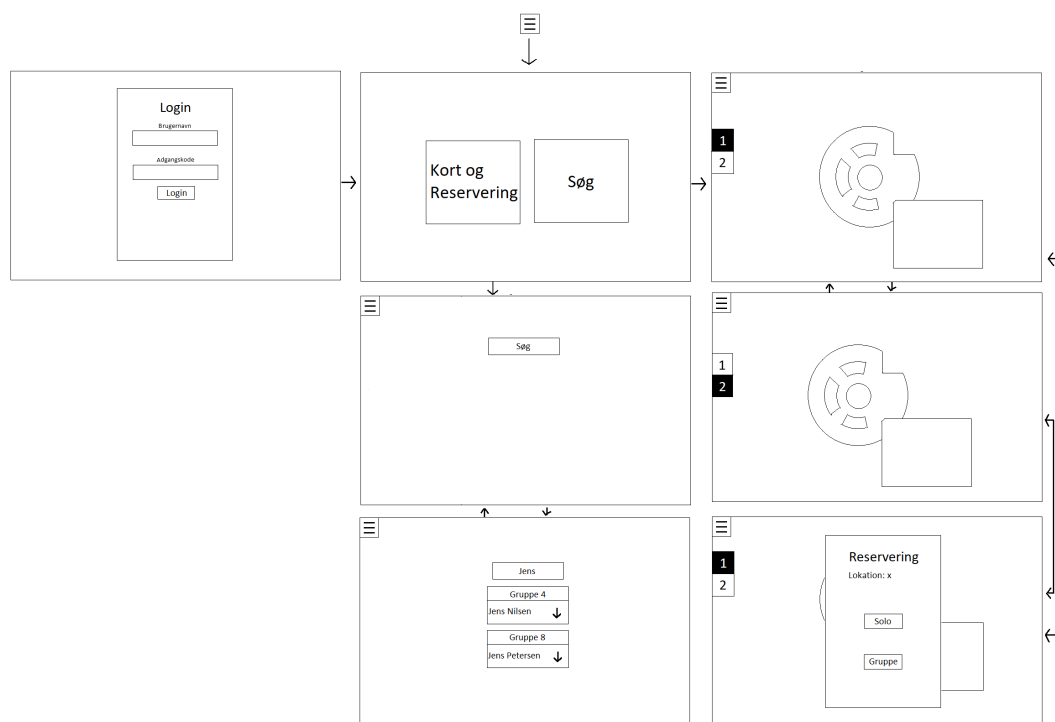
team og se hvor de har reservation for dagen. "Profil" ville give brugeren mulighed for at se deres egen information, hvad end det så måtte ende med at indeholde, og "Grupper" ville vise brugeren en oversigt over deres egen gruppe, hvem den består af og hvor - eller om - de er reserveret. Pilene der stikker ud af siderene indikere blot at den side pilen ender i kan tilgås af den hvor pilen starter. Under funktionsmenu siden ses relationen mellem burgerikonet og funktionsmenuen: at man kommer tilbage til den, hvis man trykker på ikonet på hvilken som helst side.

Efter at vi fik færdiggjort vores sketch, begyndte vi at diskutere hvad der kunne forbedres. Ydermere fik vi vist den til vores kontaktperson, som kom med feedback til hvad der kunne forbedres samt hvad der fungerede godt. Med hvad vi selv vurderede skulle ændres, samt hvad vores kontaktperson mente fungerede godt, fik vi dannet to yderligere lo-fi tegninger, som ses forneden.



Figur 8: Lo-Fi 2

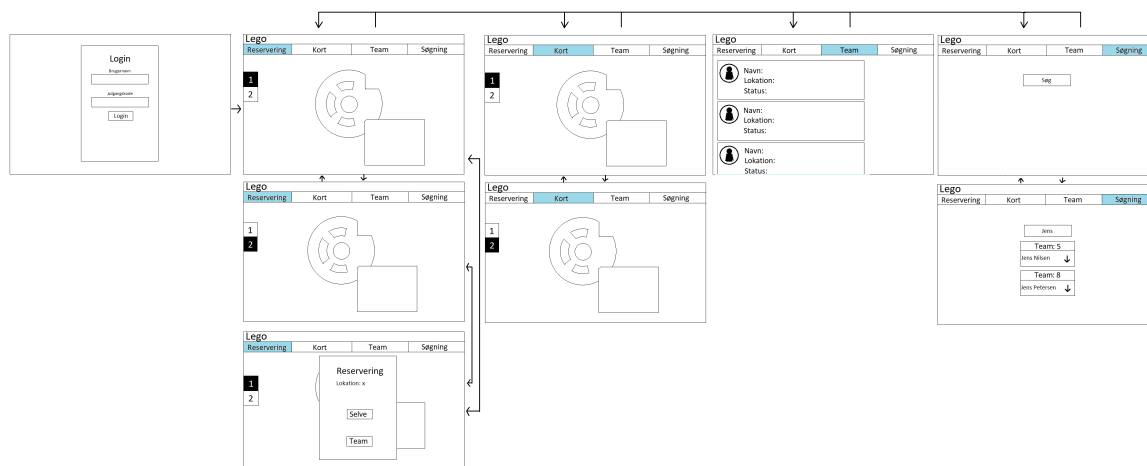
Figur 8 viser vores forslag til, hvordan vi ville designe systemet, mens Figur 9 viser den samme iteration, men med feedbacken fra vores kontaktperson inkluderet.



Figur 9: Lo-Fi 3

Begge sketches deler to ændringer fra den første lo-fi sketch, hvilket er en ny, eller ændret funktionsliste og at der ikke længere eksisterer en signup funktion. Design beslutningen om fjernelsen af signup funktionen blev vedtaget efter vi indså at det ikke som sådan gav mening at nye brugere kunne lave deres eget login, idet at antallet af brugere, og hvem der bruger systemet er besluttet af firmaet. Det ville så være firmaets opgave at lave og tildele login til deres medarbejdere. Vi vurderede derudover hvad systemet ville have brug for, og besluttede os for at udarbejde et udkast til systemet der kun havde de yderst nødvendige funktioner, i den forstand at vi ville fjerne dem vi vurderede som værende mindre vigtige eller tildels overflødige. Systemet skulle være simpelt og nemt at bruge, men simpel kan nogle gange holde samme betydning som minimalistisk, hvilket er grunden til, at vi fjernede de overflødige funktioner. I begge versioner ses kun siderne til "Kort og Reservation" og "Søg", men måden hvorpå de tilgås er forskellige. Vi foreslog at systemet kun ville inkludere en enkelt side med et søge ikon, der ville åbne en sidebar til søgefunktionen. Firmaet var glade for funktionsmenuen, da de mente at den gav brugeren et konkret overblik over hvad systemet ville give adgang til, samt en nem måde hvorpå brugeren kunne navigere til og fra siderne.

Vi sad så med to udgaver af systemet, så vi præsenterede foran vores kontaktperson begge idéer på samme tid, både for at sammenligne deres fordele og ulemper, men også for at danne et nyt udkast til design af systemet som skulle være det endelige design. Forinden ses vores endelige udkast til en lo-fi version af, hvordan systemet skulle se ud, som ses forinden i figur 10.



Figur 10: Lo-Fi 4

Den største ændring der blev vedtaget er måden hvorpå navigationen mellem siderne sker. I stedet for en funktionsmenu, har vi tegnet en navigationsbar der indeholder links til de nu fire forskellige funktioner. På navigationsbaren er den nuværende side fremhævet af en blå farve, og fra venstre til højre ses henholdsvis funktionerne "Reserver", "Kort", "Team" og "Søg". Sammenlignet med forrige iterationer, er "Reserver og Kort" blevet ændret til "Reserver", mens "Kort" er en ny side hvor man kun kan se kortet uden nogen form for funktionalitet der tilhører "Reserver". Dette gør det nemmere at finde en specifik zone, siden kortet ikke længere er fyldt med reserveringsfunktionen. Den anden nye side der blev tilføjet giver brugerne en oversigt over kun deres egne teammedlemmer, samt om de har en reservation på kontoret. Denne funktion var med i den oprindelige lo-fi sketch, og efter at have gendiskuteret hvilke funktioner der skulle inkluderes, indså vi, at vi havde fejlvurderet hvor vigtig den kunne være: uden den, mangler systemet en konkret måde at kommunikere til brugeren på, hvor de har en reservation henne. En anden mulighed ville have været at få reservationssiden til yderligere at give status på hvor en bruger er reserveret. Umiddelbart er det en lige så valid idé, men problemet opstår under en fremtidig iteration på hele systemet. I starten implementerer vi kun en gruppereservations-funktion så brugere kun kan reservere pladser til deres gruppe, men i fremtidige iterationer vil der skulle tilføjes en enkeltreservations-funktion. Man skal så derudover have en måde at se status over én's team, i tilfælde af at der var blevet lavet sådanne enkeltreservations. Derfor vælger vi at tilføje en side på systemet som vi nu kalder for "Team", hvor brugeren kan se deres teammedlemmer, samt hvor de har en reservation eller om de har én.

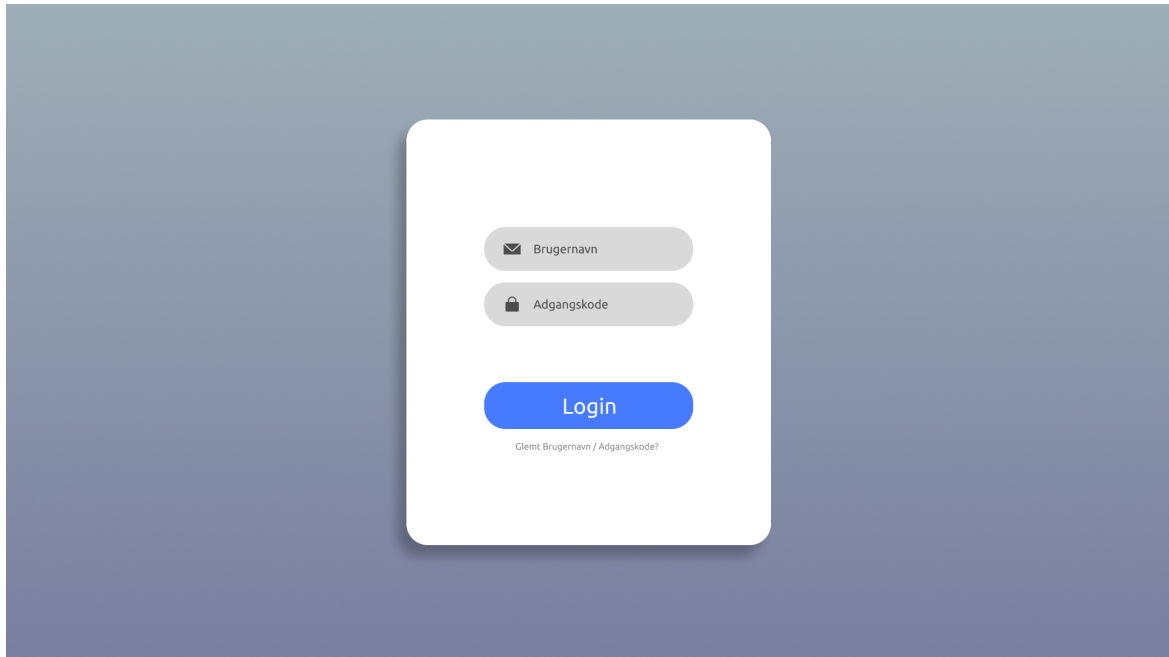
5.7 Hi-fi prototype

En "high fidelity" prototype bruges ofte til at visualisere et ordentligt designforslag under en systemudviklingsprocess. Det er et detaljeret udkast til et design af et system, hvilket det følgende afsnit vil gennemgå. Det kommer til at indeholde en oversigt over tankeprocessen bag udformningen af hi-fi prototypen samt en beskrivelse af designet. Hi-fi prototypen er baseret på og bygger videre på den seneste lo-fi prototype. Hi-fi-prototypen kommer til at agere som en vejledning under udviklingsprocessen af det endelige system, hvor der forventes at det endelige design kommer til at bære en høj grad af lighed med vores hi-fi sketch. [7]

På baggrund af den tidligere gennemførte undersøgelse, og den feedback som vi har modtaget fra vores kontaktperson under udarbejdelsen af lo-fi-prototyperne, består webstedet af følgende sider: en login side, en side hvor man kan lave reservationer, en side med kort over kontoret, en side hvor man kan se status over éns team og en side hvor man kan søge efter andre brugere eller deres team.

Under udformningen af prototypen er der blevet taget hensyn til forskellige UX-love (brugeroplevelse), hvilket har resulteret i en mere enkel og konsekvent UX. Først og fremmest er de mere almindeligt kendte Gestalt-love [8] blevet inddraget, og bruges til at forstå, hvordan brugeren opfatter de forskellige visuelle elementer på en webside, så der kan skabes et visuelt appellerende og effektivt design. Gestalt-love består af de følgende principper: nærhed, lighed, lukkethed, forbundethed & figur, hvert af disse principper, har sin egen effekt og anvendes enten alene eller i kombination med hinanden til at påvirke brugerens fortolkning. For det andet blev Jakobs lov [9] brugt til at skabe et design, som brugeren allerede er bekendt med, og Hicks lov [9] blev brugt på en måde, så siderne er simple og ligetil og dermed tilbyder et lavt antal muligheder der gør det nemmere for brugeren at vælge den funktion de vil udføre.

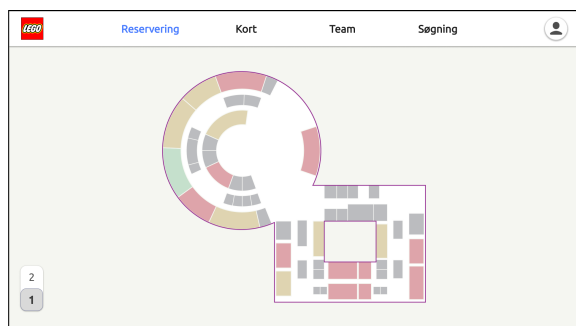
Login Side



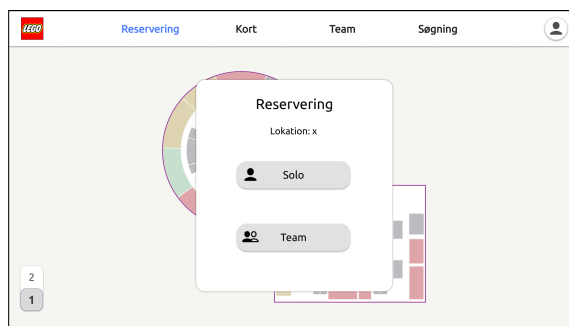
Figur 11: hi-fi Login

I de næste sektioner vil vi gå igennem alle siderne individuelt, præsentere dem og redegøre for hvorfor designvalgene blev truffet under udviklingsprocessen. Figur 11 viser login siden, som kun består af en boks med to tekstfelter, og en knap for at logge ind på systemet. Under loginknappen er der noget tekst, der agere som et link, til en funktion der kan ændre brugerens login, i tilfælde af at det bliver glemt. Vi fik overvejet om vi til at starte med ville implementere en funktion der ændre login, men vi endte med blot at inkludere det i vores hi-fi prototype som et forslag til, hvor funktionaliteten skulle sidde. Farven på baggrunden er baseret på ét af firmaets andre systemer, som gør brug af den samme farve og gradient, så ved at også gøre brug af den tilføjer vi et ekstra element af genkendelighed, hvilket derved øger kvaliteten af brugeroplevelsen.

Reserverings Side



(a) hi-fi Reservering1

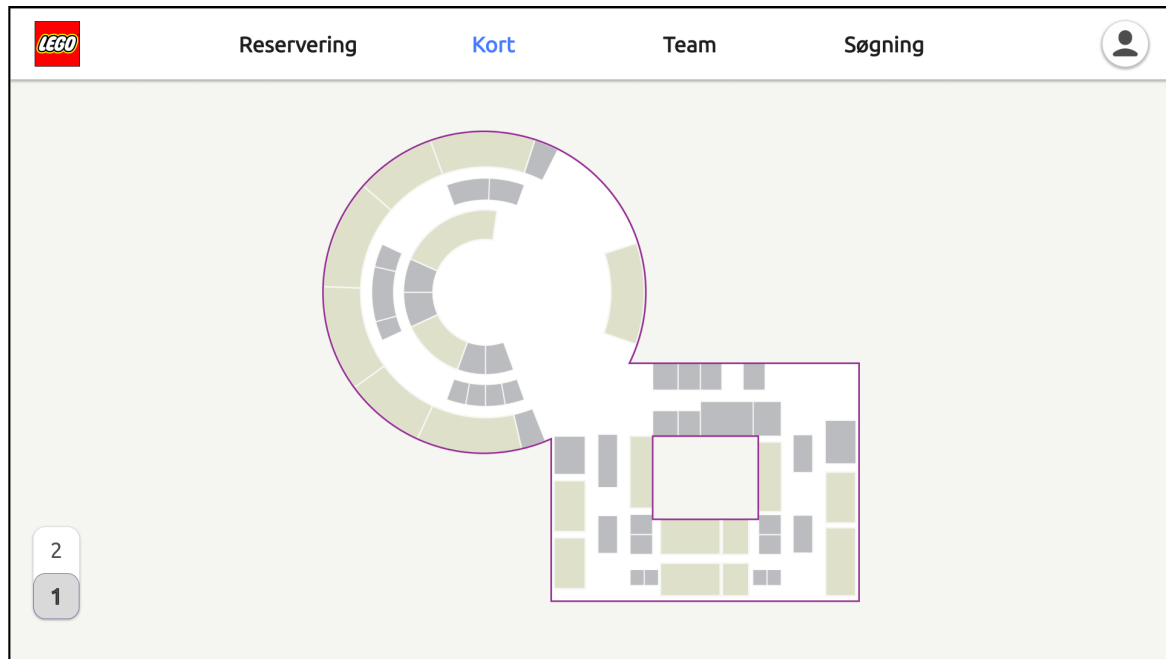


(b) hi-fi Reservering2

Figur 12: hi-fi Reservering

Reserverings siden er vist foroven på figur 12, og delt op i to dele for at vise hvordan en bruger ville interagere med systemet. Figur 12.a viser et kort over kontoret hvor alle siddepladser er delt op i områder, eller "zoner", samt farvelagt afhængig af hvor mange pladser der er ledig på et givent tidspunkt: hvis en zone er farvet grøn er alle dens pladser ledige, hvis den er farvet gul er der taget mindst én plads, og hvis den er rød er der ingen ledige pladser tilbage. Denne form for farvelægning er alment kendt blandt de fleste, nemlig idéen om at vise hvad der er tilgængeligt ved brug af farve. Oftest bliver den røde farve anvendt til at signalere til en bruger at noget er forbudt eller ikke tilgængeligt, mens en grøn farve ville signalere det modsatte. De grå områder repræsenterer lokaler i kontoret som bliver brugt til bl.a. møder, så de er ikke ledige når det kommer til en reservation gennem dette system. Derudover kan der ses hvordan en bruger skifter visning af etage nede i venstre hjørne. Kontoret har to etager, og en bruger skifter mellem dem ved blot at klikke på det nummeret af den etage, der øjeblikkeligt ikke er i brug. En bruger kan trykke på en vilkårlig zone for at åbne et reserveringsvindue, som ses på figur 12.b. Vinduet består af lidt information vedrørende tilgængeligheden af den valgte zone, samt to knapper der kan reservere en plads i zonen for enten en enkelt bruger eller deres team.

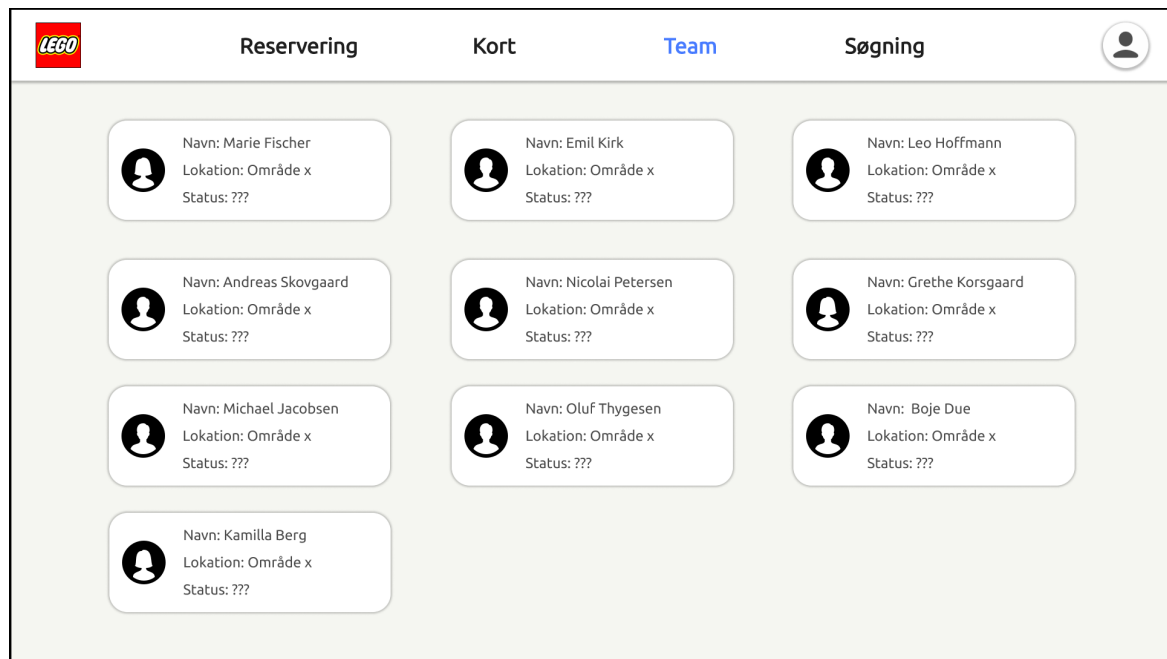
Kort Side



Figur 13: hi-fi Kort

Vist på figur 13 er et simpelt kort over kontoret som brugerne arbejder i. I modsætning til det kort der er vist i figur 12.a, er zonerne ikke farvelagt på en sådan måde, at tilgængeligheden nemt kan skues ved et øjekast, men vises i stedet bare som et normalt kort. Med denne side kan brugere nemmere se kortet uden alt den støj der opstår når man farvelægger forskelligt på et småt område.

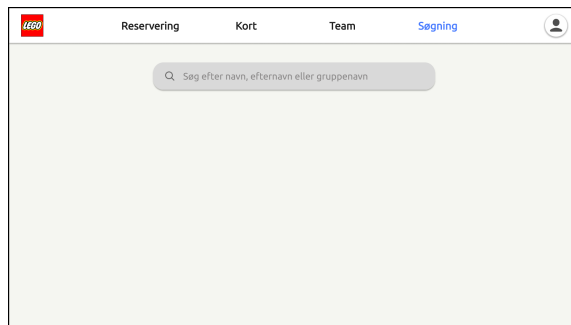
Team Side



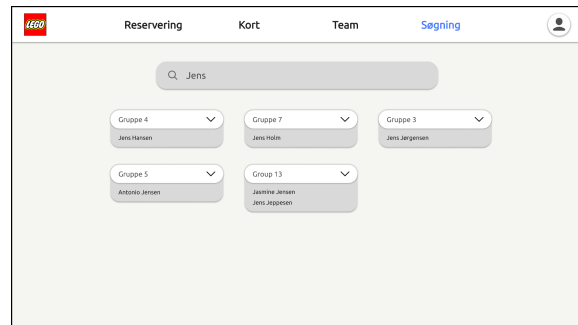
Figur 14: hi-fi Team

Forven bliver Team siden vist på figur 14. På selve sketchen er det blot en tilfældig samling af navne, samt et arbitrært antal af teammedlemmer, men under alle omstændigheder er det det samme som en bruger ville se hvis systemet bliver implementeret og brugt på arbejdspladsen. En bruger kan nemt se status over deres teammedlemmer her, samt hvor de selv kommer til at sidde for dagen, hvis én af deres teammedlemmer har reserveret deres pladser. Derudover ville brugeren også selv blive vist under listen over deres teammedlemmer. "Lokation" siger hvor brugeren har en reservation, mens "Status" siger om brugeren er reserveret eller ej. Et eksempel på det kunne være: "Lokation: Zone 4, Status: Reserveret" eller "Lokation: N/A, Status: Ikke Reserveret". Vi diskuterede senere at man alternativt kunne fjerne "Status", så den enten ville sige: "Ikke Reserveret", eller "Reserveret i Zone 5".

Søgnings Side



(a) hi-fi Søgning1



(b) hi-fi Søgning2

Figur 15: hi-fi Søgning

Den sidste side vi har designet er en side med en søge funktion, som er vist på figur 15. Figur 15.a er siden når den først tilgås af brugeren, mens 15.b er efter brugeren har søgt. Søgebaren er centreret for at sætte den helt i fokus, samt med noget tekst i søgefeltet, der informere brugeren om, at de kan søge efter navn, efternavn eller gruppe. På figur 15.b har en bruger lavet en tilfældig søgning på navnet "Jens", så de resulterende søgeresultater viser alle hændelser af navnet "Jens" i systemets model eller database bliver vist, samt yderligere hvilken gruppe de hører til.

Navigationsbar

På alle undersider bortset fra login siden, ses der en navigationsbar som består af, fra venstre til højre: et logo, fire navigationslinks til at navigere mellem de forskellige sider og endelig en knap, det tjener et teoretisk formål. Idéen med at inkludere et ekstra ikon er sådan set som et eksempel på, hvor f.eks. en eller anden form for tilgængeligheds indstillinger kunne placeres. Vi følte at det ville være en god idé allerede under udviklingen af den første iteration, at have i mente hvordan det kunne viderebygges på, så den kan i princippet ses bort fra nu. De fire navigationslinks er placeret tæt på hinanden, i henhold til lov om nærhed [9] har grupperet elementer en hyppig tendens til at være éns. I dette tilfælde deler de den samme funktionalitet, nemlig at omdirigere mellem undersiderne. Derudover anvendes en blå farveindikator til at fremhæve siden der aktuelt er i brug. Mange hjemmesider gør brug af sådanne navigationsbare, hvilket med det samme kommer til at give brugeren en følelse af kendskab, som dermed giver en bedre brugeroplevelse.

I det næste afsnit kommer vi til at gå igennem systemets udviklingsproces. Designet af systemets funktioner og UI kommer til at være baseret stort på vores hi-fi prototype, men den kommer ikke til at være Alpha og Omega. Afhængig af funktionernes aktuelle kompleksitet og den tid vi har til rådighed, kan systemet blive påvirket i overensstemmelse med det.

6 Implementering

Vores udvikling af systemet begyndte omtrent halvvejs igennem semestret. Vi havde oprindeligt tænkt os at begynde noget tidligere, men ingen af os havde hverken idé eller plan til hverken programmering eller implementering. Én af vores gruppemedlemmer fik anbefalet ”Angular” som framework, siden der angiveligt ville være mulighed for at bruge Java som sprog til nogle af delene. [10]

6.1 Design-Strategi

Før vi begyndte på at programmere vores system, diskuterede vi hvordan vi kunne styre vores udviklingsproces. Vi ville gerne fremstille en form for plan eller nogle guidelines, fordi vi mente at det ville resultere i et nemmere udviklingsforløb. Vi lavede så en liste over de funktioner der skulle programmeres og angav til dem hver især et kompleksitets niveau.

Login		Medium	Read
Vis Kort		Simple	Read
Vis Zoner		Medium	Read
Vis Gruppe		Medium	Read
Søg		Medium	Read
Reserver		Complex	Update
Afreserver		Complex	Update

Figur 16: Funktions Tabel

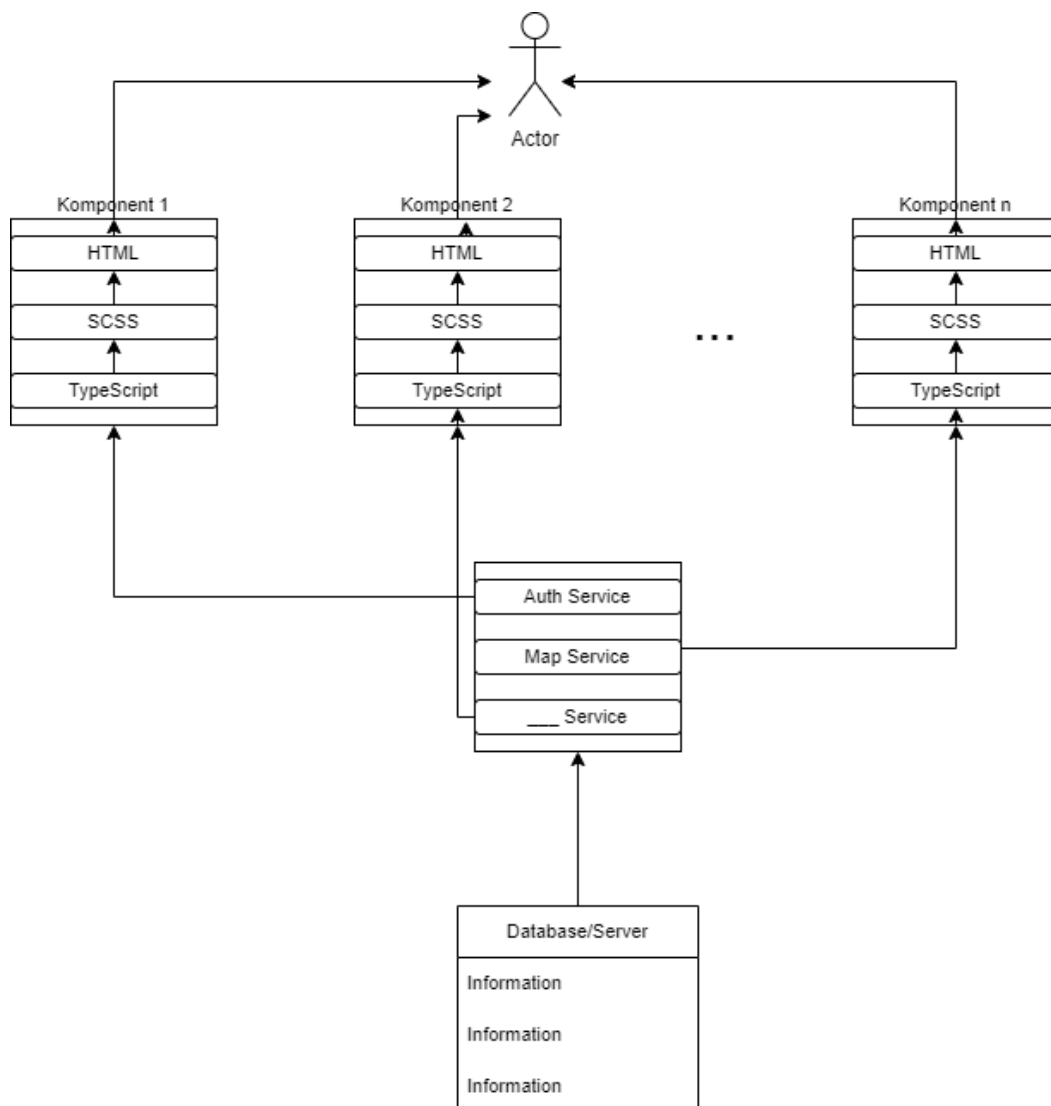
Funktionerne er samlet set forholdsvis simple, så deres kompleksitet er blevet vurderet i forhold til hinanden. Vi mener, at det at reservere og afreservere pladser er mest komplekst, da de kræver varierende bruger input, som skal ændre eller påvirke information i to dele af modellaget. På den anden side overvejede vi at starte med at lave et velfungerende login-system, der så ville tillade os at lave reserveringssystemet for at indramme systemet først. Den er vurderet som værende af medium kompleksitet da den i princippet bare skal tjekke om det login der bliver forsøgt at logge ind med er valid. At vise zonerne og grupperne vurderes som havende omkring det samme niveau af kompleksitet. De kræver et mere traditionelt brug af en ”read”funktion, da de skal hente information fra modellaget, som så bliver justeret og tilpasset af UI’en for at give brugeren præcis den information de kræver. Det samme gælder søgefunktionen. Den kommer til at tage brugerens input som en parameter, og skal så sammenligne det med hvad der er i modellaget, og så dermed vende tilbage til brugeren i UI’en med resultaterne.

Alt i alt havde vi to muligheder til rådighed: enten startede vi med at lave et velfungerende login-system, eller også skulle vi starte med at implementere en reserveringsfunktion. Begge disse tilgange ville kræve, at vi lavede brugerne i modellen først. Så under alle omstændigheder ville vi begynde med det, og hvis vi startede med at lave reserveringsfunktionen, skulle vi derudover også lave zonerne i modellen som noget af det første. Grundet gruppens mangel på erfaring med at udvikle sådan et system, blev vi enige i at finde ud af hvilken af de to der ville give mest mening, efter at vi havde fået sat vores Angular program op.

6.2 Systemet i Angular

Angular er et framework som man kan bruge til udvikling af client-server webapplikationer, som programmeres i Javascript med yderligere brug af HTML og SCSS. Noget der gør Angular specielt er i dens brug af HTML dokumenter, og hvordan der kan implementeres dynamisk funktionalitet i dem. Angular har deres egne HTML kommandoer, såsom at lave for-loop eller if-statements direkte i et HTML tag, hvilket tildels flytter funktionaliteten væk fra det tilhørende typescript dokument, og over i HTML dokumentet. Et Angular program har et hvis antal komponenter, som brugeren selv laver og ændre. Én komponent består af et HTML dokument, et SCSS dokument og et typescript dokument. Brugeren interagerer med systemet igennem HTML dokumentet, som derudover bliver påvirket af SCSS dokumentet i form af noget styling. For at få adgang til en side, inkluderes en komponent i Angulars routing dokument. I dette dokument definere udviklerne hvilken side brugeren ankommer til, når en specifik URL bliver tilgået.

Funktionaliteten som brugeren får adgang til ligger for det meste i typescript dokumentet, som derudover kan få yderligere information fra et service dokument, for at interagere med systemets modellag. En funktion i et service dokument kan indeholde f.eks. en funktion der henter information fra en database, hvilket så bliver sendt til en komponents typescript dokument, og derved hen til brugeren igennem HTML på en side de interagerer med. [10]



Figur 17: Basis Oversigt over Angular App

På figur 17 ses en oversigt over hvordan modellaget interagerer med brugeren. En service fungerer ikke nødvendigvis kun overfor én komponent, da de kan inkluderes hvor somhelst. Det er op til udvikleren selv at beslutte hvilke servicer appen skal have, samt hvilke funktioner de skal indeholde.

I vores tilfælde ville vi ende med at have seks komponenter og to services. Der ville komme til at eksistere fire sider, som enhver bruger ville kunne tilgå, samt en loginside for at validere en bruger. De ville så udgøre fem af komponenterne, hvor den sjette og sidste ville være sidens navigationsbar, som ville styre systemets routing ved at være tilstede på alle sider, bortset fra loginsiden. Derudover ville de to servicer, "authservice" og "mapservice" håndtere hhv. information fra modellaget om brugeren og information fra modellaget om zonerne.

I forhold til vores design-strategi, besluttede vi os for med Angular at lave miljøet først, som brugerene ville agere i, dvs. at vi ville starte med at lave et velfungerende login system, der så ville definere hvem brugeren var for resten af systemet. Med et valid login ville der blive sendt brugerspecifikt information til de komponenter der havde brug for brugerspecifikt information, såsom teams eller reserverings komponenterne. Loginkomponenten ville gøre brug af authservice. Den ville specialisere i alle funktioner der havde noget med brugeren at gøre i modellaget, såsom at logge ind, eller at ændre en reservation hos brugeren. Efter at systemet kunne genkende en bruger, samt håndteringen af deres information, begyndte vi på at lave reserverings- og afreserveringsfunktionen, da den havde højest kompleksitet af alle funktioner. Begge funktioner ville eksistere i den samme komponent, og gøre brug af servicen mapservice, som ville håndtere alle funktioner der havde noget med kortet i modellaget at gøre, såsom at hente alt information om zonerne fra databasen eller at opdatere antallet af ledige pladser i en zone.

Som database brugte vi en udgave af MySQL som hostes lokalt på vores computere, som hedder "MySQL Workbench" [11]. Den indeholdte to tabeller som ville blive brugt til at lagre information vedrørende alle zoner og alle brugere.

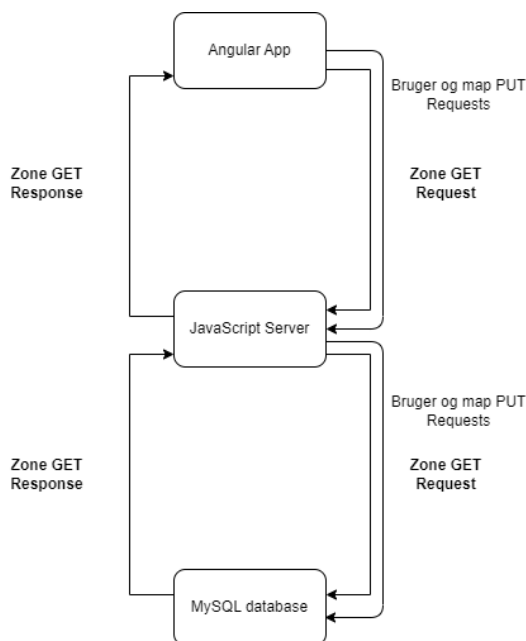
En zone havde tre værdier: et zone id som agerede som tabellens "primary key", et tal der repræsenterede hvor mange pladser en zone havde i alt og et tal der repræsenterede hvor mange ledige pladser der var tilbage i zonen.

En bruger havde syv værdier: et bruger id som agerede som tabellens primary key, et navn der inkluderede både fornavn og efternavn, et brugernavn som en bruger skulle bruge til at logge ind med og et kodeord til deres login som ikke ville være hashet eller beskyttet mere end de andre værdier. De havde også et team nummer for at identificere deres teammedlemmer og hvor mange teammedlemmer de havde repræsenteret som en integer, for at gøre det nemmere at lave en gruppe reservation, i stedet for at der skulle en ekstra funktionalitet med under reservation af zone, der kunne dynamisk finde ud af, hvor mange der var medlemmer af et team. Og til sidst, hvilken zone de er reserveret i, som ville være lig med nul som både start værdi og i tilfælde af, at brugeren ikke har en reservation.

MySQL Workbench ville blive tilgået af en backend server, der kørte på samme tid som Angular programmet, men bare i en anden port. Denne server var programmeret i Javascript, og ville styre hvilken information som servicerne ville få, afhængig af, hvilken request der blev lavet.

Hvis en tabel skulle opdateres, ville den få brugerspecifikt information fra en komponent, såsom hvilket team der reserverede en plads, eller hvilket team der skulle vises på teamsiden. I tilfælde af en reservation, ville komponenten give f.eks. teamnummer "3" samt zonennummer "6" til en service som parametre, som så derved ville lave en HTTP-PUT-request til backend serveren ved at tilgå den URL "**localhost:3000/map/3,6**", som serveren var hostet på. Den ville lave en yderligere request for at opdatere reservationen hos brugeren i modellenet ved at sende de samme parametre, men fra en anden service i Angular, til en anden sektion af backend serveren med URL'en "**localhost:3000/auth/3,6**".

Lig yderligere mærke til, at det ikke er en GET-request der bliver lavet, men en PUT-request der blot opdatere tabellen. I reserveringskomponenten under reserveringsfunktionen bliver der kaldt en funktion igen, nemlig den der henter alt informationen om zonerne. Så med det samme der sker en opdatering i tabellerne igennem en ny reservation, bliver den nye information hentet via en anden funktion. Forneden ses en grafisk visualisering af begge PUT requests, samt den yderligere GET request.



Figur 18: Basis Oversigt over Angular og Server

6.2.1 Hvorfor vi kasserede Angular Systemet

Grundet dårlig kommunikation mellem gruppemedlemmerne og forståelse for systemets overordnede krav fra et læreingsperspektiv, havde vi en diskussion om systemets langt inde i udviklingsprocessen, og om vi overhoved skulle fortsætte med det.

Som forklaringen foroven om systemet i Angular måske har hentydet til, er systemets struktur kompliceret på et unødvendigt niveau. Derudover møder programmet ikke de faglige programmeringskrav, da det hverken gør brug af Java eller objektorienteret programmering. Visse designvalg fandt vi ud af, at vi havde svært ved at forklare og forsvare, såsom hvorfor systemet ikke kunne nøjes med en direkte forbindelse mellem Angular appen og MySQL databasen.

Fremfor at prøve at at redde systemet eller at finde på forklaringer på de sære designvalg, tog vi en enorm risiko i stedet, i form at forlade systemet vi havde udviklet i Angular. Under udviklingen af Angular appen havde vi fundet et andet framework som vi mente passede meget bedre til dette projekt, da det støtter Java og objektorienteret programmering. Og netop fordi vi med det samme ville have et nyt grundlag for et nyt system, samt et forbedret udblik på de overordnede krav som systemet skulle opfylde, blev vi enige om, at forlade vores Angular App, og begynde udviklingen af det nye system.

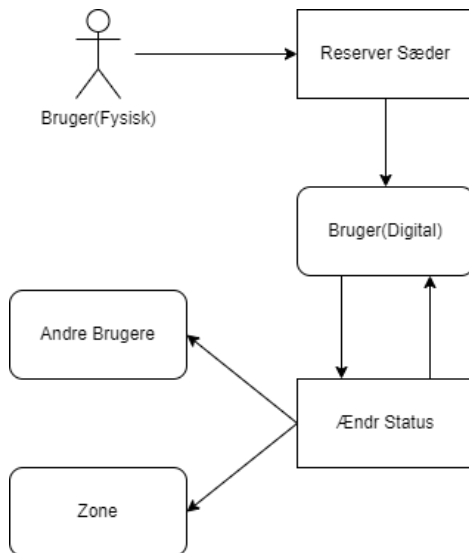
6.3 Systemet i Spring

Spring virker meget ligesom Angular på det punkt at der er controllers der styrer hele routing af hjemmesiden, når du anvender nogle af funktionerne i programmet. Der hvor det afviger, er når man skal sætte komponenter op til Angulars frontend, så har de lidt af en objektorienteret tilgang, hvor Spring anvender HTML, CSS og JavaScript i stedet som så er mere imod imperativ programmering. Så på selve systemnavigeringen er de meget ens, ellers kan det godt differere en del, dog er det Spring selvfølgelig egnede til Java og har en objektorienteret tilgang, som forekommer i alle funktionerne og den måde de er skrevet på.

En objektorienteret tilgang er når alle funktioner og alt igangsættelse er skrevet ind i klasser. De her funktioner kan så godt blive kaldt fra klasserne, men med yderligere mulighed at skabe et objekt ud fra selve skabelonen som en klasse har, så alt objekt skal kunne er skrevet inde fra klassen i forvejen, samt alle de variabler den har. Det smarte ved det er så at man kan lave flere objekter ud fra samme klasse, som så kan interagere med hinanden eller med noget helt tredje der selvfølgelig også forekommer i programmet.

I vores tilfælde er en objektorienteret tilgang meget smart, da vi kan lave brugere som har deres egen data, og zoner med deres data. Dette kan bruges på den måde at vi kan definere på forhånd hvad brugeren skal kunne, samt hvad den skal kunne interagere med, baseret på brugerens tilstand som et objekt i systemet.

Når man for eksempel gerne vil ændre egenskaber på nogle af objekterne der er pålagt serverens data, beder man enten objektet selv at lave om på det, eller et andet objekt om at sende dens data til det første objekt, illustreret forneden.



Figur 19: Brug af objektorienteret programmering

Så hver gang at en funktion bliver kaldt, er det ikke én der sidder tilfældigt, så den tankegang man skal have til dette framework er fuldt ud objektorienteret. Det vil sige at de funktioner

du kan tilgå er kun dem der står i en bruger klasse, hvori din data derudover også bliver gemt.

Med det sagt, kan vi gå tilbage til det vi har optegnet så fint tidligere i vores modeller, og egentlig tilegne det den samme logik som vi havde til design at systemet. Alt som brugeren skal kunne kalde fra klientsiden skal skrives i en brugerklasse, og alt andet der bliver ændret fra serversiden, som nok primært kommer til at være ændring af data, kommer til at skrives i andre klasser, hvilket i vores tilfælde ville være en zone klasse.

[12]

7 Kode

I dette afsnit vil vi fremvise nogle eksempler på vores kode, som bliver anvendt til at forklare hvordan vi har eksekveret den fulde implementation til systemet selv.

7.1 Introduktion af Spring

Spring giver en omfattende programmerings- og konfigurationsmodel til moderne Javabaserede virksomhedsapplikationer - på enhver form for implementeringsplatform. Et nøgleelement i Spring er infrastruktursupport på et applikationsniveau: Spring fokuserer på "installationen" af virksomhedsapplikationer, hvilket giver teams mulighed for at fokusere på forretningslogik på applikationsniveau uden at være bundet til et specifikt implementeringsmiljø.

7.2 Brugergrænsefladen

I Spring er brugergrænsefladen typisk bygget ved hjælp af en kombination af HTML, CSS og JavaScript. Disse teknologier bruges til at skabe websider, der kan gengives i en webbrowser.

For at oprette en brugergrænseflade til en Spring-Boot-applikation kan du bruge ethvert frontend-framework eller bibliotek, du foretrækker, såsom React, Angular eller Vue.js. Du kan også bruge skabelonmotorer som Thymeleaf eller FreeMarker til at generere HTML på serversiden og overføre det til klienten.

For at betjene brugergrænsefladen kan du bruge en statisk filserver eller en webserver som Apache Tomcat, som er inkluderet i Spring-Boot-startpakken. Man kan også bruge Spring MVC til at bygge en RESTful webservice og betjene brugergrænsefladen gennem den.

Når brugergrænseflade er blevet konfigureret, kan man bruge den til at sende HTTP-anmodninger til serveren for at hente data eller udføre handlinger, såsom oprettelse, læsning, opdatering eller sletning af information i en database. Serveren kan derefter besvare disse anmodninger og sende den relevante data eller statuskode tilbage.

Overordnet set er brugergrænsefladen i en Spring-Boot-applikation kun en del af den overordnede arkitektur, og den fungerer sammen med backend-tjenesterne og komponenterne for at give en komplet og funktionel webapplikation.

De valg vi så har taget indenfor inkluderinger af ekstra midler, er at bruge størstedelen af Springs egne plugins i forhold til at sætte websiden op. Dette inkluderer Thymeleaf, så man nemt kan have templates, der sætter HTML siderne op med, login system sådan så det bedst oversatte godkendelsessystem er i brug og den sidste er Spring JPA som er en database connectivity. På den måde bruger man en connection til MySQL fra Java funktionerne selv.

Vi har anvendt bootstrap CSS, som en hjælp til skalering af hjemmesiden til anvendelse af smartphones. Dette hjælper sågar også med at gøre det pænt i én hurtig aktion.

7.3 Funktionslaget

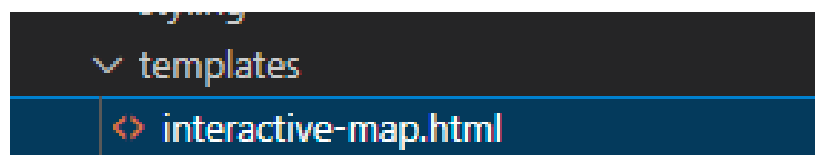
Funktionslaget starter med at være tæt ved brugergrænsefladen, da alt navigering af websiden sker via routing igennem controllers, som giver de rigtige outputs baseret på det ønskede valg taget af klienten.

```
@Controller
public class MapController {

    @GetMapping("/map")
    public String interactivemap(Model model) {
        return "interactive-map";
    }
}
```

Figur 20: Routing Eksempel 1

Foroven ses vores controller, og den styrer egentlig bare outputtet som den re-router til efter man er blevet linket til en side. I vores eksempel: "localhost:8080/map" så bliver man sat hen på en HTML template som hedder interactive-map.html:



The image shows a file explorer interface. At the top, there is a dropdown menu with a downward arrow and the text 'templates'. Below this, a list of files is shown. The file 'interactive-map.html' is highlighted with a blue background and a red icon to its left.

Figur 21: Routing Eksempel 2

Alle routes er imellem en statisk hjemmeside som i vores - og de fleste's tilfælde - er index.html, og templates som man kan se på figuren der agerer som en mappe for alle hjemmesider der forekommer herefter.

7.3.1 Funktionskald

De her controllers bruges også til funktionskald, for eksempel når vi skal tilføje noget til databasen, såsom:

```
@Controller // This means that this class is a Controller
@RequestMapping(path="/admin") // This means URL's start with /demo (after Application path)
public class MainController {
    @Autowired // This means to get the bean called userRepository// Which is auto-generated by Spring, we will use it to handle the data
    private UserRepository userRepository;
    private ZoneRepository zoneRepository;

    List<User> users;
    List<Zone> zones;

    @PostMapping(path="/adduser") // Map ONLY POST Requests
    public @ResponseBody String addNewUser (@RequestParam String firstName, @RequestParam String lastName, @RequestParam String email, @RequestParam String pass) {
        // @ResponseBody means the returned String is the response, not a view name
        // @RequestParam means it is a parameter from the GET or POST request

        User n = new User();
        n.setFirstName(firstName);
        n.setLastName(lastName);
        n.setEmail(email);
        n.setPassword(pass);
        userRepository.save(n);
        return "Saved user";
    }

    @PostMapping(path="/addzone") // Map ONLY POST Requests
    public @ResponseBody String addNewZone (@RequestParam Integer zoneNumber, @RequestParam Integer seats) {
        // @ResponseBody means the returned String is the response, not a view name
        // @RequestParam means it is a parameter from the GET or POST request

        Zone z = new Zone();
        z.SetNumber(zoneNumber);
        z.SetSeatsCount(seats);
        zoneRepository.save(z);
        return "Saved zone";
    }
}
```

Figur 22: Controller anvendelse

Her bliver det brugt til at tilføje både en ny bruger og zone til databasen, sådan så den konstruerer et nyt objekt ud fra en klasse som har egenskaber, som så gemmes til et "repository" som egentlig bare er en del af databasen.

Dette bruger vi til at skabe objekterne, som enten er brugere eller zoner, så vi netop kan interagere med deres variabler, der skal bruges til at booke og ændre status på hvor folk sidder.

Det bliver brugt som en POST funktion, hvor man så skal inkludere de parametre man giver den efter, så den så kan oprette et objekt ud fra det input den bliver givet. Det skal siges at alle funktioner bliver kaldt ud fra det objekt der lige er blevet lavet, hvilket vil sige at objektet i princippet selv ændrer disse værdier.

```
@Entity // This tells Hibernate to make a table out of this class
public class User {
    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private Integer userId;

    private String firstName;
    private String lastName;

    private String email;
    private String password;

    public Integer getId() {
        return userId;
    }

    public void setId(Integer id) {
        this.userId = id;
    }

    public String getName() {
        return firstName + lastName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}
```

Figur 23: Objektet der har funktioner

I figur 23 ses det, at funktionerne er skrevet i objektet selv, og at de bliver ændret via referencen til den selv.

Hvert objekt der bliver genereret har et unikt ID, hvilket i dette tilfælde er `userId`, som egentlig bare inkrementere per objekt type af den samme salgs.

7.3.2 Ændring af data

```
public void reserveSeat(Integer teamNumber, Zone zone) {  
    //Grab seat class minus that with the available seats in the zone  
    zone.ReserveSeatsAvailable(teamNumber);  
}  
}
```

Figur 24: En funktion under objektet User

Her i figur 24 er en funktion der sidder i klassen "User". Den bliver brugt til at kalde en funktion i "Zone", som i tilfældet af et kald kun skal bruge et team nummer, eller team størrelse. Funktionen i Zone har to parametre. Den har brug for antallet af brugere i et team, samlet den zone der bliver reserveret i. Dette er for at den kan tilrette antallet af ledige pladser i den pågældende zone korrekt.

```
public void ReserveSeatsAvailable(Integer amount) {  
    this.available -= amount;  
}
```

Figur 25: En funktion under objektet Zone

7.4 Modellaet

Når vi nu har gennemgået hvordan funktionerne kommunikerer på tværs af programmet, kan vi gå over til hvor det bliver gemt henne, ved tidligere kode eksempel: figur 22 viser at koden gemmer information i et repository, hvilket i dette eksempel er både user- og zonerepository.

Et repository er en underdel af selve MySQL databasen som vi gør brug af. Denne database er selvfølgelig hostet på en anden port end selve webapplikationen, og skal derfor tilgås via nogle egenskaber som bliver anvendt i denne kontekst:

```
spring.jpa.hibernate.ddl-auto=create-drop
spring.datasource.url=jdbc:mysql://${MYSQL_HOST:localhost}:3306/db_example
spring.datasource.username=
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
#spring.jpa.show-sql: true
```

Figur 26: Egenskaber til databasen

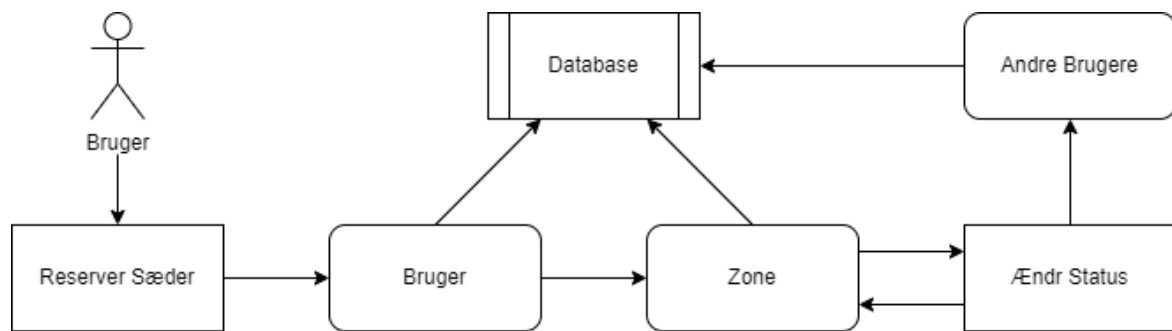
Kodestykke som kan ses i figur 26 består af enkle linjer som egentlig bare udfylder dataene som Spring frameworket skal bruge til, at få adgang til den lokale database. Det skal siges at på dette tidspunkt inden ”bygning” af programmet er den sat til at ”create-drop”, hvilket vil sige at den opretter en ny database, og sletter den per gang at applikationen kører, hvor vi helst vil have den til at være i tilstanden ”update”, så den gemmer på dataene den får, såsom zoner eller nye brugere. Vi vil så redegøre lidt for, hvordan systemet snakker med databasen og hvordan den for fat i datbasen repositories.

```
// This will be AUTO IMPLEMENTED by Spring into a Bean called userRepository
// CRUD refers Create, Read, Update, Delete
@Repository
public interface UserRepository extends CrudRepository<User, Integer> {
    List<User> findByLastName(String lastName);

    User findById(long id);
}
```

Figur 27: Repository under database

Her i figur 27 ses et repository, og måden hvorpå den fungerer er, at den bliver genereret som et tabel i databasen, så den har en helt underkategori for sig selv. Et repository kan inkludere funktioner, hvor man i dette tilfælde kan bruge det til at, kigge igennem en liste af repositories efter et bestemt efternavn eller id. Så denne del sidder ikke direkte på databasen, men det hele gemmes som en underdel til databasen.



Figur 28: Processen af reservering, men med databasen

For igen at gå tilbage til tidligere brug af visualisering - se figur 19 - så hver gang at der sker en ændring, bliver det reelt set bare gemt oppe i databasen med det samme, da det er der alle objekter i princippet eksisterer. Det vil sige at hver gang at der sker et funktionskald i systemet, kan alle med adgang til websiden få adgang til den seneste data, så brugeren vil altid være oplyst om ny ændringer i modellaget.

8 Testing

Til sidst i udviklingsprocessen foretog vi en brugertest med vores kontaktperson som den eneste deltager. Vi overvejede hvordan vi kunne udføre en ordentlig test af systemet der kunne give os brugbare resultater, da vi ikke havde ressourcerne til en effektivitetstest: for at teste hvor godt systemet ville virke i praksis, ville vi være nødt til at teste det i praksis, da det er specificeret til ét kontor, hvis ansatte allesammen ville være nødt til at deltage i testen. Det ville ikke kunne testes på samme måde på et lavere niveau, så vi besluttede os for at undlade testing af funktionerne, og hvor godt de fungerede i praksis. Alt i alt, var de glade for hvor enkelt systemet så ud, samt de funktioner der var stillet til rådighed. Hvor simpelt og visuelt programmet var, var vigtigt for dem og et stort plus.

I stedet valgte vi at lave en brugbarhedstest. Vi ville stille vores kontaktperson nogle spørgsmål angående designet og brug af systemet, og vedtage ændringer i designet på baggrund af resultaterne. Derudover ville vi stille spørgsmål angående yderligere funktioner de mente ville være brugbare, i kontekst af det næsten færdiglavet system, samt potentielle ændringer og forbedringer til de allerede eksisterende funktioner.

Først og fremmest, som en meget universel ændring mente de, at systemet et eller andet sted skulle informere brugeren om hvordan de skal interagere med systemet. Dette var mest til hvad de forskellige farver betød og om man skal trykke eller hover med musen over zonerne. De foreslog at inkludere en lille kasse et sted, der ikke ville forstyrre strukturen af funktionaliteterne, såsom i et af hjørnerne. Derudover mente de at farverne der indikerer zonens tilgængelighed ikke var synlige nok, og at de hvide cirkler kunne blive fyldt helt ud af farven.

I forhold til hvordan systemet meddeler ændringer i modellen, mente de at der skulle være en meddelelse om, at en brugers reservation er lykkedes. Dette ville være i form af f.eks. en pop-up efter de har reserveret, som enten ville være placeret i midten af siden eller som en notifikation i toppen af siden. De nævnte også, at de ville fortrække hvis systemet gav information om brugere på en lidt mere naturlig måde. Som søgeresultat og under éns teamoversigt, stod der "null" under en brugere, hvis ikke de havde en reservation. Dette mente vores kontaktperson skulle ændres til en kort sætning der ville lyde i retning af "Ingen Reservation" eller "Er Hjemme".

Da vi præsenterede systemet, var der en vigtig funktionalitet der uheldigvis manglede. Der var ikke implementeret en afreserveringsfunktion, hvilket der blev påpeget hurtigt. Derudover foreslog de en forbedring til reserveringsfunktionen, der ville indebære en tilpasning af hvem reservationen ville gælde for. I tilfælde af at syv ud af otte gruppemedlemmer skulle arbejde fysisk, mens den sidste enten var syg eller skulle arbejde hjemmefra, skulle der være en funktionalitet tilstede, der kunne give brugerne mulighed for at fravælge gruppemedlemmer, der ikke skulle have reservationen. Uden den, skulle de syv gruppemedlemmer i princippet lave deres egne enkeltreservationer, hvilket ville underminere idéen bag gruppereservationer. Til sidst nævnte de, at systemet ville øge dens brugbarhed og effektivitet ved at få tilføjet en form for kalender. Denne kalender ville give brugeren mulighed for at reservere pladser på forhånd for fremtidige dage, fremfor kun at kunne gøre det den nuværende dag. I hvilken grad de skulle have mulighed for at reservere forud var de usikre på, men mente alligevel at det ville være en god tilføjelse at overveje.

9 Diskussion

9.1 Resultat af Testing

Grundet hvor sent vores testing blev udført, var vi nødt til at fravælge nogle af de ændringer vi skulle vedtage efter testingen var færdig. Vi har vurderet hvilke ændringer vi vil lave på baggrund af hvor vigtige vi mener de er, samt hvor tidskrævene de vil være at implementere.

Af de brugergrænseoverflademæssige ændringer mente vi, at dem om farverne og information angående en brugers reserveringssatus var vigtigst. Det simple design var en vigtig del af systemet for os og kontaktpersonen, og en del af det var farverne der angiver zone tilgængelighed. At man kan få en vag men sand idé om en zones tilgængelighed ved et øjekast, mener vi er enormt vigtigt i konteksten af systemet. Derfor, hvis det på noget plan ikke virker optimalt, er det noget af det vigtigste vi vil forbedre.

Hvis systemet skal fortælle en bruger om en anden brugers status, mener vi at det skal gøres så klart og nemt som muligt. At systemet meddeler til brugeren at en anden bruger har status "null" er lidt i modstrid med denne idé, og er derfor en af de ting vi ville ændre først. Udover at være en vigtig ændring, er det en relativ nem ændring at lave.

Den eneste funktion vi ville tilføje fra vores testing er afreservationsfunktionen. Den har altid været på vores liste af vigtige funktioner, og var uheldigvis ikke implementeret da testingen blev udført. Derfor, under alle omstændigheder, vil denne funktion blive tilføjet.

9.2 Perspektivering til Arbejdsspørgsmål

I begyndelsen af rapporten opridsede vi en liste af arbejdsspørgsmål, som vi løbende ville sigte efter at få besvaret. I dette afsnit vil vi snakke om hvordan vi løste de enkelte spørgsmål, samt vurdere om det var en vellykket løsning, eller hvad vi kunne have gjort anderledes.

- Hvilken slags metode skal der gøres brug af for at give en mere strømlinet pladsfordeling? Skal man kunne reservere sin plads for en given dag, eller skal alle have faster pladser der evt. en gang imellem bliver byttet rundt?

Næsten med det samme vores problemformulering blev defineret, kom vi i gang med at diskutere hvilket slags system det skulle være, og fandt hurtigt frem til, at det skulle indebære reservering af pladser. Meget hurtigt blev det derudover fastlagt, at brugeren ville have fuld kontrol over, hvilken plads de fik. Vi ville under ingen omstændigheder involvere tilfældighed på noget plan, så et system der ville tildele pladser med en grad af tilfældighed havde vi ingen planer om at lave. Senere blev idéen om simplicitet til én af kerneegenskaberne bag systemet. Idéen bag denne simplicitet var, at det ville gøre det nemmere og hurtigere for brugerne, at reservere deres pladser, men vi føler lidt at vi ikke gik langt nok med det. Udover at gøre designet simpelt, har vi ikke gjort så meget andet for at øge brugervenligheden. Derudover er designet for det meste baseret på idéen om genkendelighed, idet at de fleste funktioner oftest nemt ville kunne genkendes af brugerne, men det er ikke sikkert med garanti, at alle ville kunde gøre brug af systemet med samme niveau af effektivitet, da der i princippet er en mangel på nogen form for vejledning.

- Hvordan skal medarbejderene kunne tilgå programmet? Skal det være en online service, eller et program man skal installere, enten på egen computer eller fælles computer?

Fra da vi begyndte at definere rammerne for implementation af systemet, havde der været snak om hvorvidt det skulle være en mobilapp eller ej. De krav som vi fik fra vores kontaktperson specificerende bl.a. at det helst skulle kunne tilgås fra en telefon, og igennem diskussion blandt gruppen, blev det vedtaget at mobiltelefonstilgængelighed ville være et "should have", ifølge vores MoSCoW. Det er blevet nævnt før, bl.a. under både MoSCoW analysen og PAKT analysen, at hvis systemet skulle blive brugt af kontoret, ville det kunne tilgås igennem en mobiltelefon.

Det endelige system vi så ville aflevere ville være en form for prototype af det aktuelle system, idet at det reelt set ikke kan blive brugt optimalt, da det kun er egnet til computere, og derudover ikke er hostet på en server, der kører på ubestemt tid. Som programmet er nu, kan det ikke blive brugt af et kontor, men det har vi vurderet som værende fint, da det principielt godt ville kunne implementeres, og bruges korrekt, uden megen besvær.

- *Hvilke design elementer skal der ligges mest fokus på? Gør vi eksklusivt brug af specifikke design principper, eller designeres det på vores egen gennemtænkt måde?*

Som sagt lagde vi stort fokuset i, at lave en strømlinet og hurtig reserveringsprocess. Til det formål havde vi ikke en specifik design metode, men I stedet gjorde vi brug af en samling af metoder og principper. Først og fremmest er det en bestemt baggrund vi gør brug af. Fremfor at vælge en arbitrær baggrundsfarve, foreslog vores kontaktperson at vi gjorde brug af den samme baggrund som ét af firmaets egne systemer. Vi fik tilsent et billede af en af systemets undersider, og lavede baggrundsfarven på basis af den.

Navigationsbaren som vi lavede har vi vurderet som værende et godt element i forhold til denne strømlinet brugeroplevelse som vi sigtede efter at lave. Hvorend en bruger er i systemet, kan de altid hurtigt navigere hen til hvilken en funktion de nu har et ønske om at tilgå. Vi prøvede så derudover at designe funktionernes brugergrænseoverflader med idéen om at centrere de vigtigste elementer, for at skabe den bedste sammenhæng mellem hvad funktionen er og hvordan brugeren interagerer med den.

Dette var vores mål med designet. Den er mest baseret på vage design principper, der langt fra kan omtales som specifikke, samt heller ikke anses som værende frygteligt gennemtænkte på nogle planer. Hvis målet var at lave et brugervenligt design, kunne vi godt have designet systemet på en mere specifik måde. Med det menes der, at i stedet for at basere det delvist på nogle designprincipper, såsom centrering, der er vage men omfattende, kunne vi havde designet det ud fra designprincipper der er mere specifikke, og som ville påvirke designet samlet set i en højere grad.

- *Hvordan sikrer man sig, at programmet fungerer som kunden ønsker? I hvor stor en grad skal testing køre? Skal der laves kvalitativ testing eller kvantitativ testing?* Igennem vores testing ved vi at vores kontaktperson er tilfreds med produktet. I hvilken grad kan vi ikke sige på nuværende tidspunkt, da vores testing ikke kunne køre i en stor nok grad til at det kunne vurderes ordentligt. Vi fik kun udført én udgave af kvalitativ testing, og havde ikke mulighed for at udføre nogen form for kvantitativ testing. Vi har så dermed ikke kunne kombinere de to for at forme en velstruktureret, udarbejdet og informativ testing, hvilket i sidste ende betyder, at vi er usikre i systemets reelle effektivitet: vi ved ikke, og har ingen måde hvorpå vi kan vide om kunden ville være veltilfreds med systemet.

9.3 Forbedringer på Systemet

I dette afsnit vil vi se bort fra de overvejelser vi har gjort os i det forrige afsnit, og i stedet fokusere på de forbedringer vi kunne lave på det nuværende system. Til det formål vil vi inddrage vores MoSCoW model, og diskutere forbedringer med den i mente. Vi kommer til at diskutere de yderligere lavt vurderet krav, og hvad det ville indebære for systemet hvis de blev virkeliggjort.

Vi fik implementeret alle vores Must have's i systemet, men det var trods alt det eneste vi kunne nå. Vi var enige om, at den første yderligere funktionalitet systemet skulle have efter alle Must have's var blevet opfyldt, var mobiltelefonstilgængelighed, som vi allerede har diskuteret i det forrige afsnit. Udover det, ville vi ændre på søgefunktionen sådan, at en bruger ville kunne søge efter et domain eller et subdomain, fremfor kun at kunne søge efter et team eller navn. Som systemet ser ud nu, ville det kræve en markant ændring i klassernes struktur hvis domains og subdomains skulle tilføjes. Vi har dermed vurderet den tilføjelse som værende mindre vigtigt, dels grundet dens implementation, men også at behov for den er stortset baseret på spekulation. Derfor, ville der ikke være noget, som vi ville tilføje først.

Som en del af vores Should have's, havde vi vurderet sikkerhed som værende mindre vigtigt til dette system, da et systembrug ikke ville have hverken store- eller langtidskoneskvenser. Men med det sagt, ville sikkerhed selvfølgelig værre en velkommen funktionalitet i systemet. I forhold til hvordan man ville sikre systemet, kunne man tilføje en to-faktorgodkendelse igennem enten de ansattes email eller telefonnummer.

Et andet forslag til funktionalitet var at tilføje såkaldte flyverpladser til kortet, hvis en medarbejder kun havde brug for en siddeplads i kort tid. Reelt set ville det være en okay tilføjelse at lave, men vi har indset at det underminere lidt hvad det vil sige, at tage en flyverplads. Meningen med de pladser er jo, at de er frit tilgængelige til alle. Så idet man inkorporerer dem ind i et reserveringsystem, mister de lidt deres formål. Derfor har vi besluttet os for, ikke at lave denne funktionalitet hvis vi kunne videreudvikle systemet.

Systemets kort over konteret er hårdtkodet ind i programmet, så hvis der er behov for en ændring i kortet skal det ændres direkte i koden. Til dette formål, ville systemet have en ekstra slags bruger, nemlig en form for administrator, der kunne ændre på kortet og zonerne. Vi fik diskuteret senere hen, at denne administrator derudover også kunne stå for brugeroprettelse på selve systemet. Så fremfor at indsætte et nyt login direkte i databasen, kunne denne administrator tilgå en brugeroprettelsesfunktion på selve systemet for dermed at lave nye brugere. Hvilke andre rettigheder de skulle have, kan vi ikke vurdere på basis af denne version af systemet. Afhængig af hvilke problemer brugerne kommer til at støde på efter systemet er blevet implementeret korrekt og er i brug, ville en administrator kunne løse eller afhjælpe så godt som de nu kan, og på nuværende tidspunkt, kan vi ikke sige hvilke slags problemer det ville komme til at være.

Det eneste punkt på vores Wont have var tilføjelsen af andre lokationer end LEGO's kontor i Billund. Principielt er der intet i vejen med at realisere denne funktion, men den ville nok kræve en meget stor ændring i systemet, samt en undersøgelse i om der reelt set er behov for sådan et system på kontoret i København, på samme niveau som dette. Systemet ville

skulle akkommodere for to lokationer, samt sikre sig at en bruger fra Billund under ingen omstændigheder ville kunne tilgå funktionerne fra systemet i København. Der skal ydermere overvejes om det skal være to forskellige systemer, hvis deres behov afviger for meget fra hinanden. Det kan være en vigtig funktionalitet at inkludere, men resten af systemet skal færdiggøres inden den her begynder at blive udviklet.

9.4 Retrospektive Overvejelser

I dette afsnit vil vi påpege de største fejl vi begik under udviklingsprocessen i lyset af, at vi snart er færdig med projektet. Vi kommer ikke til at nævne noget for specifikt, men i stedet hvad vi kunne have gjort anderledes, som vi mener ville samlet set have forbedret projektets kvalitet.

Vi mener at den største fejl vi begik under projektet var, at vi var for ivrige til at begynde med programmering af systemet. Nogen af os har hidtil oplevet under andre projekter, at nogle af projektets største utilstrækkeligheder havde oprindelse i hvor sent programmeringen begyndte. Alle i gruppen var med på at begynde på at programmere så hurtigt som muligt, men fordi der var usikkerhed i hvordan og hvorledes vi ville styre programmeringsforløbet, endte det ultimativt i at vi lavede to udgaver af vores system. På grund af denne mangel på retning, var der ingen af os der stillede spørgsmålstejn ved valg af programmeringsframework, så vi begyndte at udvikle Angular systemet uden at sikre os, at det var det rigtige valg.

I begyndelsen af projektet skulle vi i kontakt med en ekstern enhed, og vores fremgangsmåde til at starte med bestod af, at sende en masse e-mail til forskellige firmaer og grupper. Mens vi ventede svar på dem, gjorde vi ikke brug af den tid vi fik allokeret til projektarbejde. Der var gået knap tre uger fra projektets opstart til vi begyndte at arbejde på problemet. I lyset af, hvordan projektet har forløbet sig, mener vi at de tre spildte uger kunne have blevet brugt mere fornuftigt. Vi diskuterede hvordan vi kunne have brugt tiden bedre, siden vi dengang havde hverken problem eller retning at gå efter. Hvad vi kunne have gjort anderledes spiller sammen med de forrige punkt i dette afsnit, nemlig en mangel på både retning og huller i forståelsen for hvad projektet gik ud på. Vi skulle have brugt tiden på universel forberedelse af projektet, fremfor at vente på at vi kunne forberede i kontekst af et problem. Vi kunne have undersøgt mere på, hvad projektet samlet set gik ud på, hvad det endelige system skulle indebære eller forberede arbejdesgangen og arbejdsmiljøet.

10 Konklusion

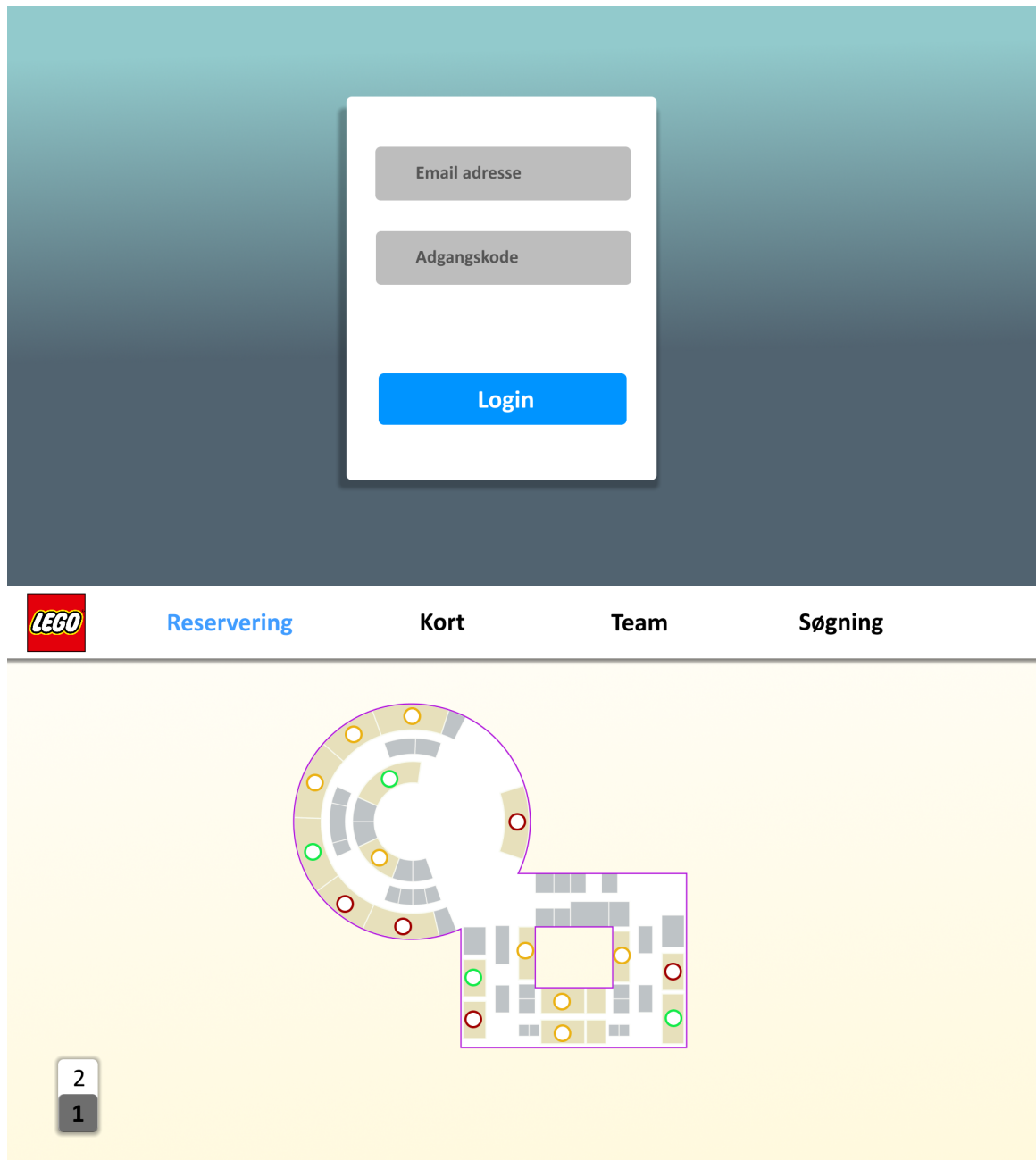
Alt i alt var udviklingsprocessen af systemet fuld af uheldige hændelser forårsaget af, at vi ikke havde forberedt os nok. Vi endte med at fremstille to forskellige udgaver af systemet, hvor kun den ene havde opfyldt de udviklingsmæssige krav. Igennem vores brugertest fik vi yderligere foreslag til forbedringer af systemet, men grundet tiden der blev spildt på, at lave systemet i et forkert udviklingsmiljø, havde vi ikke nok tid til at implementere alle foreslag.

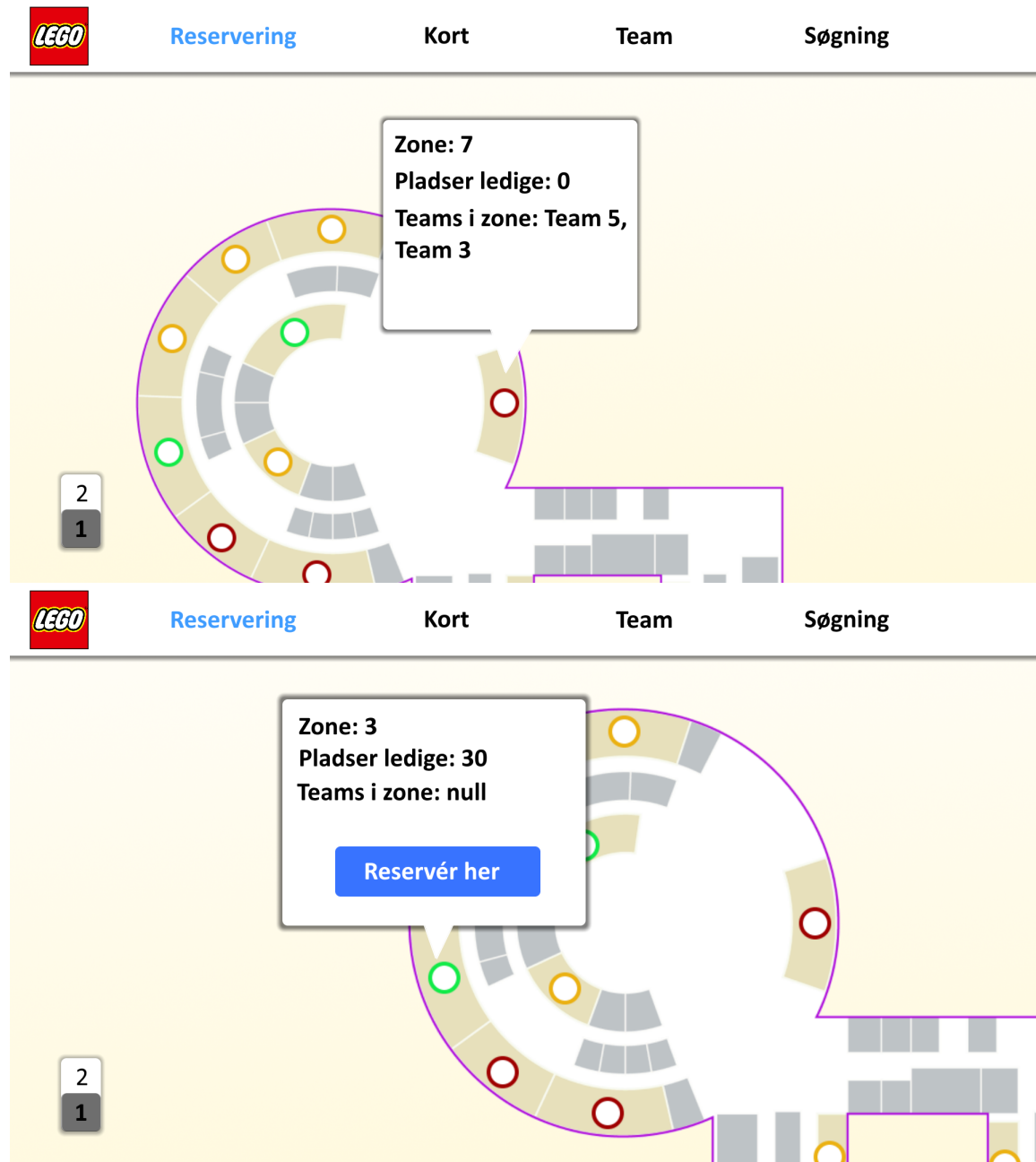
Grundet systemets natur som en reserveringsservice, ville en kvantitativ undersøgelse i systemets effektivitet være umulig at udføre. I stedet måtte vi nøjes med en test af systemets brugergrænseoverflade, hvilket resulterede i de førnævnte forbedringer vi implementerede. Så selvom systemets brugergrænseoverflade blev testet, og vurderet som værende et vellykket design, kan det samme ikke siges for funktionerne, udover i en teoretisk forstand. Derudover er der stadig potentielle funktioner som kunne tilføjes til systemet, så i tilfælde af, at systemet skulle viderudvikles, er det klart for os, hvad vi ville tilføje og forbedre.

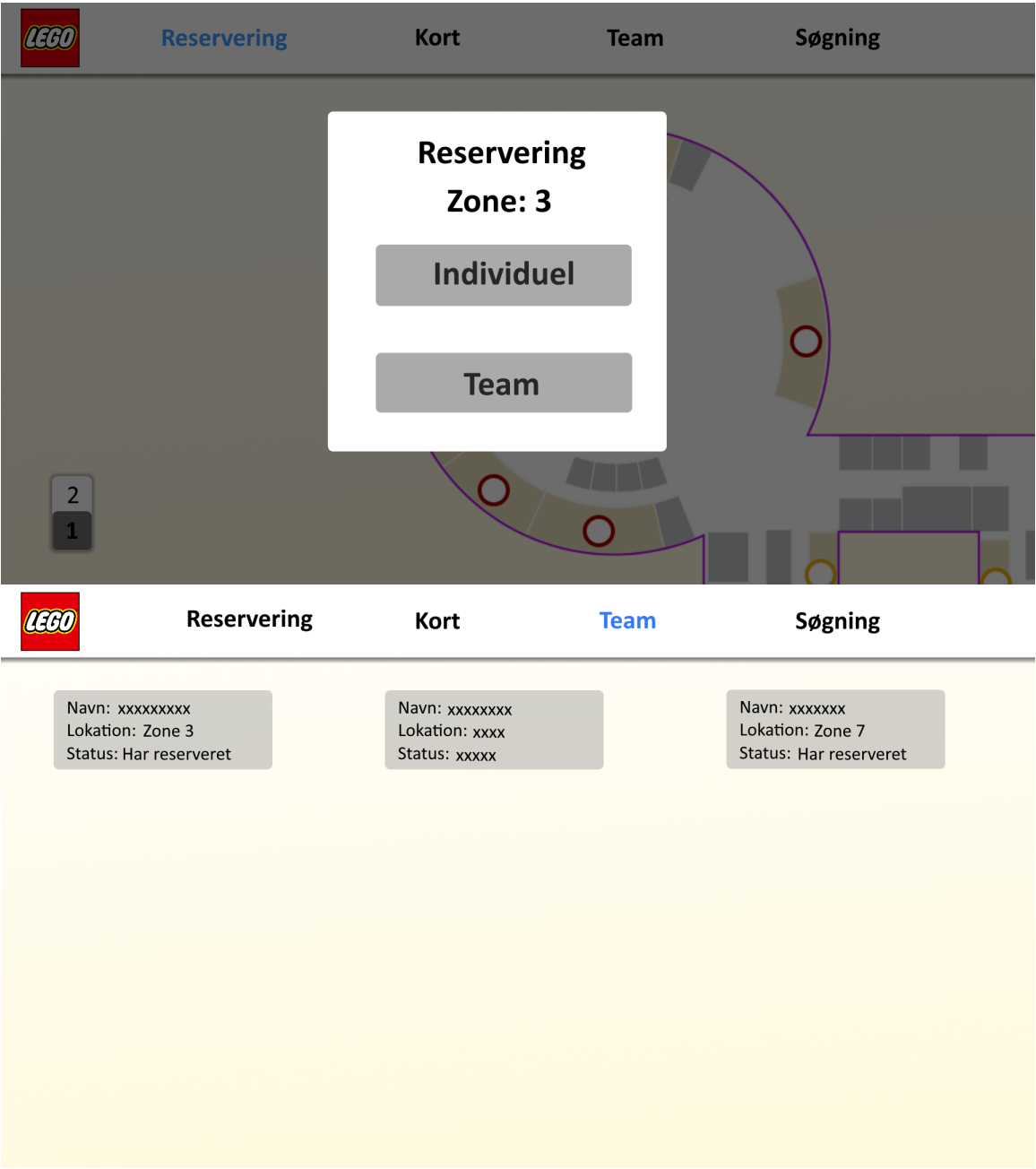
De krav der blev sat for os af både projektet og firmaet til at starte med blev opfyldt, men udover dem, blev der ikke tilføjet så meget mere. På baggrund af det, kan man i en teoretisk forstand så konkludere, at systemet løser problemet om LEGOs kontorpladser, men i en usikkergrad af effektivitet.

Litteratur

- [1] BetterEvaluation. *Rich pictures*. 2022. URL: <https://www.betterevaluation.org/methods-approaches/methods/rich-pictures>. (accessed: 15.12.2022).
- [2] Lars Mathiasen Andreas Munk-Madsen Peter Axel Nielsen Jan Stage. *Object-Oriented Analysis and Design*. Marko ApS, 2001. ISBN: 87-7751-153-0.
- [3] Product-Plan. *MoSCoW Prioritization*. 2022. URL: <https://www.productplan.com/glossary/moscow-prioritization/>. (accessed: 12.10.2022).
- [4] Rashmini Nayanathara. *PACT Analysis*. 2021. URL: <https://bootcamp.uxdesign.cc/pact-analysis-3ac5fbe8817>. (accessed: 15.12.2022).
- [5] David Benyon. *Designing User Experience fourth edition*. Pearson Education Limited, 2019. ISBN: 978-1-292-15551-7.
- [6] Figma. 2022. URL: <https://www.figma.com/>. (accessed: 16.12.2022).
- [7] Bartłomiej Pierzchała. *Low Fidelity vs High Fidelity Prototypes*. 2018. URL: <https://medium.com/7ninjas/low-fidelity-vs-high-fidelity-prototypes-903a7befaa5a>. (accessed: 15.12.2022).
- [8] Multimediedesigneren. *Gestaltlovene: Vigtige principper i design*. 2022. URL: <https://multimediedesigneren.dk/gestaltlovene/>. (accessed: 16.12.2022).
- [9] Alina Khazanova. *20 Principles of Website Design Every Web Professional Should Know*. 2022. URL: <https://elementor.com/blog/principles-of-website-design/>. (accessed: 5.12.2022).
- [10] Angular. *Introduction to the Angular docs*. 2022. URL: <https://angular.io/docs>. (accessed: 16.12.2022).
- [11] MySQL. *MySQL Documentation*. 2022. URL: <https://dev.mysql.com/doc/>. (accessed: 16.12.2022).
- [12] Spring.io. *Why Spring?* 2022. URL: <https://spring.io/why-spring>. (accessed: 16.12.2022).









ReserveringKortTeamSøgning

Søg efter fornavn, efternavn eller gruppe



ReserveringKortTeamSøgning

Brian

Gruppe 5

Brian jørgensenZone: 5

Brian beckHjemme